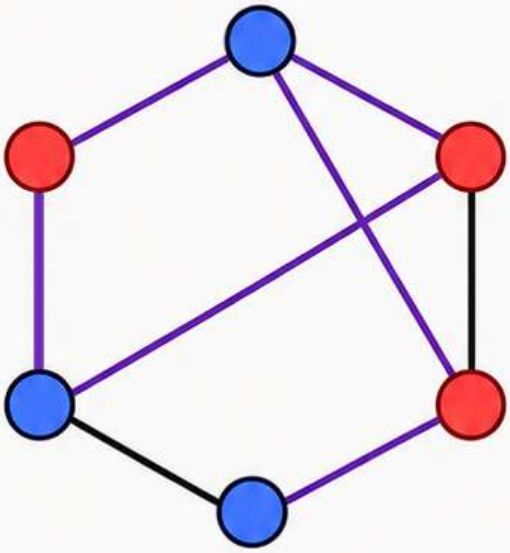
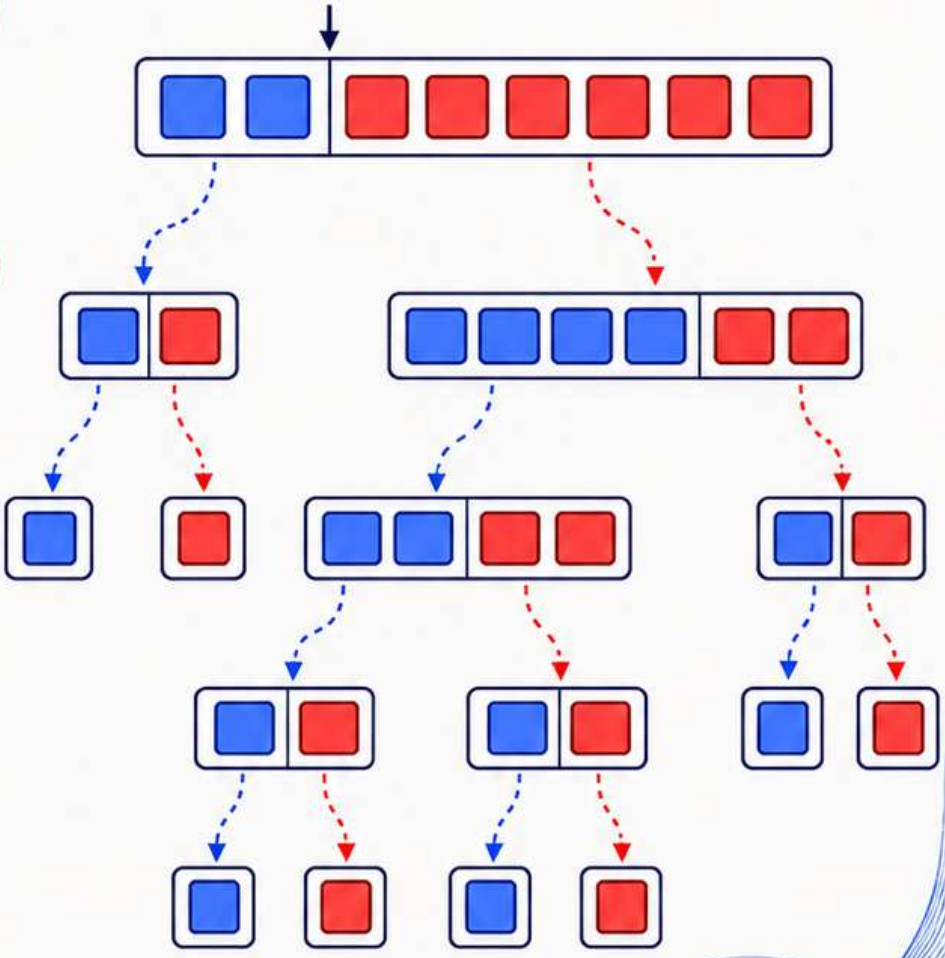
# Randomised Algorithms

Sahil M Deo





# Contents

# Chapter 1

# Introduction

## 1.1 What is a Randomised Algorithm?

A randomised algorithm is an algorithm that makes random choices during its execution. Unlike deterministic algorithms, whose behaviour is completely determined by the input, the behaviour of a randomised algorithm may vary between different executions on the same input.

Typically, the algorithm uses a random number generator (`rng`) to decide between multiple possible actions. As a result, quantities such as running time[1], memory usage, or even correctness[2] can become random variables (though usually not all of them at the same time).

The performance of a randomised algorithm is therefore analysed using probability and expectation. If $T_{run}$ denotes the runtime of the algorithm, we are usually interested in quantities such as $T_{algo} = E[T_{run}]$ over all possible random choices made by the algorithm.[3]

Randomised algorithms are often used for the following purposes:

- avoid adversarial worst case inputs (common in competitive programming)
- simplify algorithm design and implementation
- obtain polynomial-time approximate solutions to NP problems (the field of heuristic algorithms)

## 1.2 Implementation of Randomness in `C++`

Let us make things more concrete with some actual code and implementation details! `C++` provides pseudo-random[4] number generators through the `<random>` library. A commonly used generator is `mt19937`, which is based on the Mersenne Twister algorithm.

[1] Such algorithms are called Las Vegas algorithms.

[2] Such an algorithm is called a Monte Carlo algorithm. The fact that correctness itself may become probabilistic can initially seem undesirable, especially in critical systems where absolute correctness is expected. However, in many practical situations, a sufficiently small error probability is considered acceptable. For example, if an algorithm can be shown to fail with probability at most $10^{-30}$, then the chance of failure may already be smaller than the probability of random hardware faults such as memory bit flips caused by cosmic radiation, which programmers typically ignore during ordinary software development.

[3] $T_{algo}$ can still be a random variable over the input - so this may **not** be the same as average case performance of the algorithm $T_{avg}$, which is an expected value over all possible inputs. But for cleanliness of presentation, we will often ignore this distinction and use these terms interchangeably or simply use the variable $T$ to mean either of them, depending on context.

[4] Since computers are deterministic, they rely on parameters like atmospheric noise and temperature for randomness. The details are irrelevant for us... We will take pseudo random as true random for our purposes

```
#include <bits/stdc++.h>
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()); //using time as a seed
```

This creates an object rng with seed[5] as current time which will be passed as a parameter to other functions. It has the capability to produce "truly" random 32-bit integers.

### Generating Random Integers

To generate a random integer uniformly from a range $[L, R]$, we use

```
uniform_int_distribution<int>uid(L,R);

int x=uid(rng);
int y=uid(rng);
.
.
.
```

Each generation takes around constant time.

### Generating Random Real Numbers

To generate a real number uniformly from an interval $[x, y)$, we use

```
uniform_real_distribution<double>urd(x,y);

double x=urd(rng);
double y=urd(rng);
.
.
.
```

This also takes constant time per generation.

### Generating a Random Permutation

A random permutation of an array (in-place modification) can be generated using `shuffle`:

```
shuffle(a.begin(),a.end(),rng);
```

Internally, this does something like:

```
for(int i=n-1;i>0;i--)
{
    int j=random integer from [0,i];
    swap(a[i],a[j]);
}
```

Deduce why all permutations are equally likely with this method!

[5]Seed values are used in pseudo random generators as a way to recreate the generated random numbers. This can be used for debugging a randomised algorithm. If you don't care about that, set it to the current time in milliseconds or something similar and that should be good enough!

### Implementing Random Decisions

Sometimes, an algorithm must choose between multiple actions with specified probabilities. For example, let three actions X,Y,Z be such that they are executed with probabilities:

$$\Pr(X) = \frac{1}{5}, \qquad \Pr(Y) = \frac{3}{5}, \qquad \Pr(Z) = \frac{1}{5}$$

One simple way to implement this is to generate a random integer uniformly from 1 to 5:

```
uniform_int_distribution<int>uid(1,5);

int v=uid(rng);

if(v==1)
{
    //do X
}
else if(v<=4)
{
    //do Y
}
else
{
    //do Z
}
```

thus implementing the required probabilities.

## 1.3 Problems

**1** The following procedure is applied on an array $a$ of $n$ integers –

```
for(int i=n-1;i>0;i--)
{
    int j=random integer from [0,i];
    swap(a[i],a[j]);
}
```

**(a)** Show that this indeed generates a permuation.

**(b)** Show that every permutation is equally likely to be generated.

**2** Suppose you have access only to a function `bool coin()` which returns 0 or 1 with equal probability. Let the phrase "generate a probability $p$" mean designing a function which returns 1 with probability $p$ and 0 with probability $1-p$.

**(a)** Design a procedure to generate a probability $\frac{k}{2^m}$ using only finite number of calls to `coin()`.

**(b)** Design a procedure to generate any rational probability $\frac{p}{q}$ with **expected** finite number of calls to `coin()`
*Hint: Think of the binary representation of a number.*

**(c)** Generalise the above ideas to generate any real probability $p$ with **expected** finite number of calls to `coin()`.

# Chapter 2

# Algorithmic Performance

For algorithms whose performance[1] depends on the input size or other input parameters, we usually estimate the number of elementary operations performed as a function of the input size $n$ or the maximum value of input $A_{max}$, usually denoted by $A$. This gives a mathematical way to compare algorithms independent of hardware or implementation details.

A classical example is `BubbleSort`. The algorithm repeatedly traverses the array, comparing adjacent elements and swapping them whenever they are in the wrong order. After each pass, the largest remaining element moves to its correct position near the end of the array. With some thought it can be proved that the total number of swaps performed by `BubbleSort` is exactly equal to the number of inversions $N_{inv}$ in the input array. (An inversion is a pair of indices $(i, j)$ such that $i < j$ but $a_i > a_j$.) By taking swaps to be one operation taking constant time, we can estimate runtime of `BubbleSort` to be $T_{run} \propto N_{inv}$

## 2.1 Worst Case Analysis

In worst case analysis, we determine the maximum possible number of operations over all valid inputs of size $n$. For `BubbleSort`, the worst case occurs when the array is sorted in reverse order, where every pair is inverted.

$$N_{inv,max} = \binom{n}{2} = \frac{n(n-1)}{2}$$

Worst case analysis is useful because it guarantees an upper bound on the running time regardless of the input provided.

## 2.2 Average Case Analysis

In average case analysis, we compute the expected number of operations assuming that all valid inputs are equally likely. For `BubbleSort`, this corresponds to analysing the expected number of inversions in a random permutation of size $n$, where all $n!$ permutations are equally likely. Define the indicator[2] variable $I_{ij}$ for each pair $(i, j)$ as

$$I_{ij} = \begin{cases} 1, & \text{if } (i,j) \text{ is an inversion} \\ 0, & \text{otherwise} \end{cases}$$

[1] Although this section focuses primarily on running time, the same methods of analysis can also be applied to other resources such as memory usage.

[2] Refer Appendix A.6

The point of defining the indicator variable is to notice that the number of inversions is the sum of all $I_{ij}$. Formally we can write this as

$$N_{\text{inv}} = \sum_{1 \le i < j \le n} I_{ij}$$

For every pair of indices $(i, j)$ with $i < j$, the probability that the pair forms an inversion is $\frac{1}{2}$ since for every permmutation with $a_i > a_j$, there is a permuation with $a_j > a_i$ and vice versa.

$$\Pr\big((i,j) \text{ is an inversion}\big) = \frac{1}{2} \implies \mathbb{E}[I_{ij}] = \frac{1}{2}$$

Using linearity of expectation[3],

$$\mathbb{E}[N_{\text{inv}}] = \mathbb{E}\left[\sum_{1 \le i < j \le n} I_{ij}\right] = \sum_{1 \le i < j \le n} \mathbb{E}[I_{ij}] = \sum_{1 \le i < j \le n} \frac{1}{2} = \frac{n(n-1)}{4}$$

A valuable strategy which we can deduce from this analysis is to randomise the input sequence. In case of an adversary[4], this can be useful because we can expect an improvement in time by factor of 2.

## 2.3 Time Complexity Analysis

Let us first define the asymptotic complexity of a function $f(n)$ in terms of $n$. My favourite way to think about complexity is that we try to find the "simplest" function $g(n)$ such that

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = k, \quad k \in \mathbb{R}_{>0}$$

This is compactly represented as $f(n) = \Theta(g(n))$. If via inequalities we are able to find $f(n) \le h(n)$ or $f(n) < h(n)$ and $h(n) = \Theta(g(n))$, then we can represent that by $f(n) \le \Theta(g(n))$. Similarly we define what it means to say that $f(n) \ge \Theta(g(n))$.

By applying Sandwich theorem on the ratio $f(n)/g(n)$, we can conclude that if $f(n) \ge \Theta(g(n))$ and $f(n) \le \Theta(g(n))$ then $f(n) = \Theta(g(n))$.

Since "simplest" function is not really well defined, it is best to look at some examples and gain intuition!

| Function | Asymptotic Complexity |
|---|---|
| $7$ | $\Theta(1)$ |
| $n^2 + 100n + 1$ | $\Theta(n^2)$ |
| $n + \log n$ | $\Theta(n)$ |
| $\log(n!)$ | $\Theta(n \log n)$ |
| $\sum_{i=1}^{n} i$ | $\Theta(n^2)$ |
| $\sum_{i=1}^{n} \frac{1}{i}$ | $\Theta(\log n)$ |
| $\sum_{i=1}^{n} \log i$ | $\Theta(n \log n)$ |

[3] Refer Appendix A.1

[4] An adversary is someone who deliberately gives inputs to make your algorithm perform badly. Randomising the input is one possible defense against adversaries.

In the example of `BubbleSort`, the worst case runtime was $T_{run}(n) = \frac{n(n-1)}{2}$. So you will often come across statements like "`BubbleSort` has worst case time complexity $\Theta(n^2)$". What this means is that the complexity of the worst case runtime as a function of $n$ is $\Theta(n^2)$.

A natural question which arises in comparing different algorithms for the same task is which one is faster? For example, consider two algorithms A and B, where A has runtime $T_{run}(n) = n^2$ and B has runtime $T_{run}(n) = 100n \log n$. To simplify our comparison, we just compare their complexities.

Comparing between the complexity functions $f(n)$ and $g(n)$ themselves is quite straight forward – we just evaluate the limits of the ratio $f(n)/g(n)$ as $n \to \infty$. If the first limit is 0, then $f(n) < g(n)$, if the limit is a non-zero constant (usually 1) then $f(n) = g(n)$, otherwise $f(n) > g(n)$. Basis these simple rules, we can have a total order of the common complexity functions as

$$\Theta(1) < \Theta(\log n) < \Theta(n^{\frac{1}{x}}) < \Theta(n) < \Theta(n \log n) < \Theta(n^2)$$

In the above comparison, $x > 1$. Algorithms having time complexity smaller than $\Theta(n)$ are called sub-linear time algorithms.

Finally all the above discussion allows us to say that A: $T_{run} = n^2$ is worse than B: $T_{run} = 100n \log n$ since A is $\Theta(n^2)$ and B is $\Theta(n \log n)$ even though for small n the first one may well be better, but for small n the performance doesn't really[5] matter anyway...

[5]Integer multiplication is an interesting case. The classic method of multiplying digit by digit as taught in school is $\Theta(n^2)$ time, but there are complicated methods which can do it in $\Theta(n \log n)$ time. But the benefit of using the complicated methods really doesn't show up until quite a large n because of high constant factor.

## 2.4 Problems

**1** A `vector` in `C++` is an array with variable size. Once the memory allocated to the `vector` is exceeded, it is reallocated more memory somewhere else. The designers of the `vector class` must choose a sequence of sizes $s_0 = 0 < s_1 < s_2 < \ldots$ such that when the size of the `vector` becomes any $s_i$, we reallocate and assign memory of size $s_{i+1}$ to the `vector`.

Thus each `push_back()` operation is one operation if space is available and $s_i+1$ otherwise. For the average case analysis take all sizes of vectors between 0 and $n-1$ (both inclusive) to be equally likely.

**(a)** If $s_i = i$, show that the time complexity for the average `push_back()` is $\Theta(n)$.

**(b)** Show that in general the time complexity for the average `push_back()`is $\Theta\left(1 + \frac{\left(\sum_{s_i<n} s_i\right)}{n}\right)$.

**(c)** Knowing the previous result, come up with a simple series of $s_i$ that would give a time complexity of $\Theta(1)$ for the average `push_back()`.

**2** Consider the algorithm

```
f(n)
{
    count = 0
    while(n>1)
    {
        if(n even)
            n = n/2
        else //n is odd
            n = n-1
        count = count + 1
    }
    return count
}
```

Here $f(n)$ counts the number of iterations required to reduce the number $n$ to 1. We can think of $f(n)$ as a measure of the time taken by the algorithm (one iteration being one operation). Now consider $x \in \{2^{k+1} - 1 : k \in \mathbb{N}\}$ and $n$ be the input constrained to be between 1 and $x$. Find the asymptotic[6] ratio of worst case time and average case time of the algorithm with respect to n. Equivalently, find

$$\lim_{x\to\infty} \frac{\max_{1\le n\le x} f(n)}{\left(\frac{1}{x}\sum_{n=1}^{x} f(n)\right)}$$

[6]We have seen this ratio being 2 in the case of `BubbleSort`.

# Chapter 3

# The Classics

## 3.1 QuickSort

In general, to sort an input array of size n deterministically using only pairwise comparisons, it can be proved via a decision tree that we need at least $n \log_2 n$ operations. From a time complexity point of view this is $\Theta(n \log n)$ time. `QuickSort` does not really improve upon that, but in practice it is very quick with a small constant factor. The STL library of `C++` uses `QuickSort` in its `std::sort()` function.

### 3.1.1 The Algorithm

```
QuickSort(A)
{
    choose an element p from A as the "pivot"
    partition A into A<p, A=p, A>p
    return {QuickSort(A<p) : (A=p) : QuickSort(A>p)}
}
```

Note that `A<p` is our notation for the elements of `A` that are strictly less than `p`. Similarly we can define `A>p` and `A=p`.

### 3.1.2 Worst Case Runtime

A deterministic implementation may always choose the first or last element as pivot. Unfortunately, this can lead to very poor performance on adversarial inputs. For example, if the array is already sorted and we always choose the leftmost element as pivot, the recursion becomes extremely unbalanced –

$$(n-1) + (n-2) + \cdots + 1 = \Theta(n^2)$$

Thus `QuickSort` has worst case complexity

$$T(n) = \Theta(n^2)$$

The standard randomised version instead chooses the pivot uniformly at random. This tiny modification dramatically changes the behaviour of the algorithm. Intuitively, it becomes very unlikely that we repeatedly choose extremely bad pivots.

### 3.1.3 Expected Runtime

#### Recurrence Relation

Let $T(n)$ denote the expected runtime of randomised `QuickSort` on an array of size $n$ over all possible inputs.

Suppose the pivot finally ends up at position $k$, where $0 \leq k \leq n-1$. Then the recursive subproblems have sizes $k$ and $n-1-k$. Since partitioning itself takes n operations, we obtain the recurrence

$$T(n \mid \text{pivot} = k) = T(k) + T(n-1-k) + n$$

Because the pivot is chosen uniformly at random, every value of $k$ is equally likely. Taking expectation over all random choices made by the algorithm gives

$$E[T(n \mid \text{pivot} = k)] = n + \frac{1}{n}\sum_{k=0}^{n-1}\Big(T(k) + T(n-1-k)\Big)$$

Using the law[1] of total expectation gives $E[T(n \mid \text{pivot} = k)] = T(n)$. Using that and the fact $\sum_{k=0}^{n-1} T(k) = \sum_{k=0}^{n-1} T(n-1-k)$,

$$T(n) = n + \frac{2}{n}\sum_{k=0}^{n-1} T(k)$$

Changing the variable from $T(n)$ to $S(n)$ gives,

$$\implies S(n) - S(n-1) = n + \frac{2}{n}S(n-1) \quad (\text{Substituting } S(n) = \sum_{k=0}^{n} T(k))$$

$$\implies S(n) = \left(1 + \frac{2}{n}\right) S(n-1) + cn$$

$$\implies \frac{S(n)}{(n+1)(n+2)} = \frac{S(n-1)}{n(n+1)} + \frac{n}{(n+1)(n+2)}$$

$$\implies \frac{S(n)}{(n+1)(n+2)} - \frac{S(n-1)}{n(n+1)} = \frac{2}{n+2} - \frac{1}{n+1}$$

Summing this recurrence telescopically gives

$$\frac{S(n)}{(n+1)(n+2)} = \Theta(\log n) \implies S(n) = \Theta(n^2 \log n)$$

$$\implies T(n) = S(n) - S(n-1) = \Theta(n \log n)$$

Hence the expected runtime of randomised `QuickSort` is $\Theta(n \log n)$

[1] See Appendix A.7

**Indicator Variables**

Here we explore an alternative method to arrive at the expected runtime of `QuickSort`. Observe that any pair of elements is compared at most once during the entire execution of `QuickSort`. Without loss of generality assume the array is a permutation of $\{1, 2, \ldots, n\}$. Consider the indicator variable,

$$I_{ij} = \begin{cases} 1, & \text{if elements } i, j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

Then the total number of comparisons $C$ is given by

$$C = \sum_{1 \le i < j \le n} I_{ij}$$

Two elements are compared only if one of them becomes the first pivot chosen among all elements lying between them in sorted order. Since all of them are equally likely, we have a 2 out of $j - i + 1$ chance of this happening.

$$\Pr(I_{ij} = 1) = \frac{2}{j - i + 1} \implies \mathbb{E}[C] = \mathbb{E}[\sum_{1 \le i < j \le n} I_{ij}] = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} = \Theta(n \log n)$$

## 3.2 QuickSelect

`QuickSort` is used to Find the $k$-th smallest element[2] of an unsorted input array. An obvious method is to sort the array using `MergeSort`[3] or `QuickSort` and then access the $k$-th element in $\Theta(1)$. But since we only care about one element, it seems inefficient to sort the entire array.

### 3.2.1 The Algorithm

We pick a pivot $p$ and partition the array as before. Basis the size of the partitions we can figure out which partition the $k$-th element is in. If it is in the `A=p` partition, then we already have our answer! Else we recurse into that partition and continue this process until we find the $k$-th element.

```
QuickSelect(A,k)
{
    choose an element p from A as the "pivot"
    partition A into A<p, A=p, A>p
    if size(A<p) >= k                   f// k-th element is in A<p
        return QuickSelect(A<p,k)
    else if size(A<p)+size(A=p) >= k    // k-th element is in A=p
        return p
    else                                // k-th element is in A>p
        return QuickSelect(A>p,k-size(A<p)-size(A=p))
}
```

[2] This is commonly referred to as the $k$-th order statistic of the array.

[3] `MergeSort` is a divide and conquer deterministic sort algorithm which runs in $\Theta(n \log n)$ time.

Intuitively, this is similar to `QuickSort`. However, unlike `QuickSort`, we recurse into only one side – each step discarding a significant[4] fraction of the array. This is a similar approach to Binary Search although not quite same since this does not assume a sorted array.

### 3.2.2 Expected Runtime

Let $X_i$ denote the size of the search space after $i$ recursive calls of `QuickSelect`. ($X_0 = n$)

At each step, a pivot is chosen uniformly at random and the array is partitioned around it. Depending on the rank we are searching for, `QuickSelect` recurses into exactly one of the two partitions. To obtain a worst-case bound, we imagine that the algorithm always recurses into the larger partition and that all elements are distinct.

If the current search space has size $x$, and the pivot ends up at position $k$, then the larger partition has size $\max(k-1, x-k)$

Averaging over all pivot positions,

$$\mathbb{E}[X_{i+1} \mid X_i = x] = \frac{1}{x}\sum_{k=1}^{x} \max(k-1, x-k) \leq \frac{3}{4}(x-1) \leq \frac{3}{4}x$$

$$\implies \mathbb{E}[X_{i+1} \mid X_i] \leq \frac{3}{4}X_i$$

Taking expectations and applying the law of total expectation,

$$\mathbb{E}[X_{i+1}] = \mathbb{E}[\mathbb{E}[X_{i+1} \mid X_i]] \leq \frac{3}{4}\mathbb{E}[X_i]$$

$$\implies \frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq \frac{3}{4} \implies \frac{\mathbb{E}[X_i]}{\mathbb{E}[X_0]} \leq \left(\frac{3}{4}\right)^i$$

$$\implies E[X_i] \leq n(\frac{3}{4})^i \quad (\text{Using } \mathbb{E}[X_0] = X_0 = n)$$

Let $T$ be the random variable which denotes the number of recursive calls of `QuickSelect`. It is the first $i$ such that $X_i \leq 1$.

Taking one comparison with the pivot as one operation,

$$E[T_{run}] \leq \sum_{i=1}^{T} n(\frac{3}{4})^i \leq \sum_{i=1}^{\infty} n(\frac{3}{4})^i = 3n$$

So the expected runtime of `QuickSelect` is $\Theta(n)$, which is a significant improvement over the sort-and-access method which takes $\Theta(n \log n)$

### 3.2.3 Expected Recursion Depth

Recall that $H$ is the first $i$ such that $X_i \leq 1$. Since $X_i \leq 1$ is equivalent to $i \geq T$, $i < H$ is equivalent to $X_i \geq 2$. Using Markov's Inequality[5],

$$\Pr(i < H) = \Pr(X_i \geq 2) \leq \frac{\mathbb{E}[X_i]}{2}$$

[4] On an expected basis.
[5] See Appendix B.3

Using the upper bound for $\mathbb{E}[X_i]$,

$$\Pr(X_i \geq 2) \leq \frac{\mathbb{E}[X_i]}{2} \leq \frac{n}{2}\left(\frac{3}{4}\right)^i$$

Using the tail sum formula[6],

$$\mathbb{E}[H] = \sum_{i=0}^{\infty} \Pr(H > i)$$

$$\implies \mathbb{E}[H] \leq \sum_{i=0}^{\infty} \min\left(1, \frac{n}{2}\left(\frac{3}{4}\right)^i\right) \quad \text{(Using the trivial } Pr(H > i) \leq 1\text{)}$$

**The Splitting[7] Technique** We split the sum into two parts, and bound them or find them separately, thus finding our overall bound. Here we split according to when the min() changes from 1 to $\frac{n}{2}\left(\frac{3}{4}\right)^i$.

$$\mathbb{E}[H] \leq \sum_{i=0}^{m-1} 1 + \sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i = m + \sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i \quad \left(\text{where } m = \left\lceil \log_{\frac{4}{3}}\left(\frac{n}{2}\right)\right\rceil\right)$$

For the summation we can factor out the first term,

$$\sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i = \left(\frac{n}{2}\left(\frac{3}{4}\right)^m\right) \cdot \left(\sum_{j=m}^{\infty}\left(\frac{3}{4}\right)^{j-m}\right)$$

We have $\frac{n}{2}\left(\frac{3}{4}\right)^m \leq 1$ (By definition of $m$) and $\sum_{j=m}^{\infty}\left(\frac{3}{4}\right)^{j-m} = \sum_{j=0}^{\infty}\left(\frac{3}{4}\right)^j = \frac{1}{1-\frac{3}{4}} = 4$

$$\implies \sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i \leq 4$$

Combining the two bounds,

$$\mathbb{E}[H] \leq m + 4 = \left\lceil \log_{4/3}\left(\frac{n}{2}\right)\right\rceil + 4$$

So we can conclude $\mathbb{E}[H] \leq \Theta(\log n)$

[6] See Appendix A.2

[7] I feel this is useful and common enough to warrant a name!

## 3.3 Problems

**1** **(a)** We concluded an upper bound for the expected recursion depth $\mathbb{E}[H] \leq \Theta(\log n)$ for `QuickSelect`. But to justify the expected complexity as $\Theta(\log n)$, show that $\mathbb{E}[H] \geq \Theta(\log n)$

**(b)** For a general series of random variables $X_0 = n, X_1, X_2, \ldots$ and T being the first $i$ such that $X_i \leq 1$, given $\frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq p$ with $0 < p < 1$, show that $\mathbb{E}[T] \leq \Theta(\log n)$

**2** Let $C_n$ be the number of comparisons made by `QuickSort` on an array of size $n$. We already established $\mathbb{E}[C_n] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$

**(a)** By grouping together all pairs with the same value of $j - i$, show that

$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = 2 \sum_{k=1}^{n-1} \frac{n-k}{k+1}$$

**(b)** Let $H_n = \sum_{i=1}^{n} \frac{1}{i}$ be the $n$-th harmonic number. Using this and the above result show that

$$\mathbb{E}[C_n] = 2(n+1)H_n - 4n = \Theta(n \log n)$$

# Chapter 4

# Hashing

## 4.1 What even is Hashing?

A recurring theme in randomized algorithms is the idea of replacing a large object by a much smaller *fingerprint* or *hash* which approximately preserves equality.

Instead of comparing two complicated objects directly, we compare their hashes. If the hashes differ, the objects are certainly different. If the hashes are equal, the objects are *probably* equal.

The power of hashing comes from the fact that computing and comparing hashes is often significantly faster than comparing the original objects themselves. It may also be seen as a form of compression that preserves some information about the original object. In data structures like the `unordered set`, hashing helps to achieve constant time complexity (on average) within finite memory constraints.

Usually hashes are many-one functions, meaning that multiple different objects can have the same hash. Two inputs having the same hash is called a *collision*. The probability of collision depends on the specific hash function used and the distribution of inputs. A good hash function should minimize the probability of collisions for typical inputs.

## 4.2 Polynomial Multiplication

Suppose we are given two expressions:

$$P(x) = \prod_{i=1}^{k}(a_i x + b_i), Q(x) = \sum_{j=0}^{k} c_j x^j$$

We would like to verify whether

$$P(x) \equiv Q(x)$$

Naively expanding the product takes $\Theta(k^2)$ time, and then comparing the resulting polynomials takes $\Theta(k)$ time, so the total verification time is $\Theta(k^2)$.

A more efficient approach is to compute the hashes of the polynomials. Choose three random values

$$r_1, r_2, r_3$$

from the domain of the polynomial.

Define the hash of a polynomial to be the tuple

$$(P(r_1), P(r_2), P(r_3))$$

Similarly compute

$$(Q(r_1), Q(r_2), Q(r_3))$$

If the hashes differ at any coordinate, then the polynomials are certainly different.
If all three evaluations agree, then with high probability the polynomials are identical.

The reason this works is that the nonzero polynomial $P(x) - Q(x)$ of degree at most $k$ can have at most $k$ roots in the domain from which we are choosing the random values.

If we are choosing random 32-bit integers, then the probability of a false positive (i.e. the polynomials are different but the hashes match) is at most $\left(\frac{k}{2^{32}}\right)^3$ since we are checking three independent random values, which is astronomically small for any reasonable $k$!

Each evaluation of $P$ or $Q$ at a random point takes $\Theta(k)$ time, so the total verification time is $\Theta(k)$. So we have achieved a significant improvement over the naive[1] $\Theta(k^2)$ time.

Note that choosing 3 random values has no particular significance - we could choose any small constant number of random values to achieve this - it is a tradeoff between time and correctness probability.

## 4.3 Matrix Multiplication

Suppose we are given matrices $A, B, C \in \mathbb{R}^{n \times n}$ and wish to verify whether

$$AB = C$$

Computing the product explicitly requires matrix multiplication. Standard matrix multiplication takes $\Theta(n^3)$ time.
time, while the fastest known algorithms run in roughly $\Theta(n^{2.37})$ time, though they are highly complicated and rarely used in practice.

### 4.3.1 Freivalds' Algorithm

Choose a random vector $X \in \{0, 1, \ldots, p-1\}^n$, aka of form

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Instead of comparing the matrices themselves, we compute $ABX$ and $CX$. Since matrix-vector multiplication takes only $\Theta(n^2)$ and we are doing 3 such multiplications – $B \cdot X$, then $A \cdot (BX)$, and $C \cdot X$. Then we compare the two products entry by entry in $\Theta(n)$ time. So the overall procedure is $\Theta(n^2)$ time.

The quantity $MX$ may be viewed as a compact randomized hash of the matrix $M$. If two matrices differ, their hashes are unlikely to coincide for a random choice of $X$.
More generally, one may choose k independent random vectors

$$X_1, X_2, \ldots, X_k$$

and define the hash of a matrix $M$ as

$$\mathbf{H}(M) = (MX_1, MX_2, \ldots, MX_k)$$

Each additional random vector independently decreases the probability of error. Here k must be $\Theta(1)$ to preserve the overall $\Theta(n^2)$ time complexity.

[1] There exists deterministic techniques to get this down to $\Theta(k \log k)$ time, but they have a large constant factor and also are very difficult to implement.

### Error Probability Analysis

Let $M = AB - C$. We want to bound the probability that $MX = 0$ when $M$ is nonzero. Choose a prime $p$, and consider the finite field

$$\mathbb{Z}_p = (\{0, 1, 2, \ldots, p-1\}, +_p, \times_p)$$

To avoid making the entries of $M \equiv 0 \pmod{p}$ when it is nonzero over $\mathbb{Z}$ (and thus avoiding making it null in $\mathbb{Z}_p$), we use such a $p$ which is strictly greater than every entry of $M$. We then choose the random vector $X \in \mathbb{Z}_p^n$ uniformly at random.

Notice that whenever $MX = 0$ over the integers, we also have $MX = 0$ over $\mathbb{Z}_p$ since $0 \equiv 0 \pmod{p}$. So the set of false positives over $\mathbb{Z}$ is a subset of the set of false positives over $\mathbb{Z}_p$. Thus the probability of a false positive over $\mathbb{Z}$ is at most the probability of a false positive over $\mathbb{Z}_p$.

Now $(Z_p)^n$ is a vector space[2] of dimension $n$ over $\mathbb{Z}_p$. If $M$ is not the null matrix, the dimension of its nullspace[3] is at most $n - 1$.

A $d$-dimensional vector space over $\mathbb{Z}_p$ contains exactly $p^d$ vectors. So

$$|nullspace(M)| \leq p^{n-1}$$

Since $X$ is chosen uniformly from $(Z_p)^n$, which contains $p^n$ vectors,

$$\Pr(MX = 0 \mid \mathbb{Z}) \leq \Pr(MX = 0 \mid \mathbb{Z}_p) \leq \frac{p^{n-1}}{p^n} = \frac{1}{p}$$

A suitable choice is $p = 2^{64} + 13$ if we are working with 64-bit integers, which gives an astronomically small error probability of about $5.4 \times 10^{-20}$.

## 4.4 A Matrix Problem

**Problem Statement[4]:** Given two $n \times m$ matrices $A$ and $B$, determine if they are equivalent under row and column permutations. That is, can we permute the rows and columns of $A$ to obtain $B$?
Additional Constraints: A and B both satisfy – the set of all entries is {1,2, . . . , nm}.

**Reduction:** First we reduce the problem to a simpler one. If we consider R(M) to be the set of all $R_i$ where each $R_i$ is the set of values in the $i$-th row of a matrix M, and similarly C(M) then the matrices A and B are equivalent under row and column permutations if and only if R(A) = R(B) and C(A) = C(B).

### Proof of Reduction

Clearly, if A and B are equivalent under row and column permutations, then R(A) = R(B) and C(A) = C(B) since the row and column sets are preserved under these permutations. This proves that the set of all B such that R(B) = R(A) and C(B) = C(A) contains all matrices equivalent to A under row and column permutations (i.e. the set of all B such that R(B) =

[2]It may be helpful to brush up on the theory of vector spaces, but you may choose to not bother with the finer details and take my word for it!

[3]Refer Appendix E.4

[4]This question was taken from Codeforces. The editorial indicates that my hashing solution wasn't intended, but when has that ever stopped me?

R(A) and C(B) = C(A) is a superset of the set of all B equivalent to A under row and column permutations).

Now notice that the number of row and column permutations on A are $n! \cdot m!$ (including the identity permutation). Each of these permutations gives a different matrix since the entries are all distinct. Also it can be shown that the number of matrices with given R(M) and C(M) is also $n! \cdot m!$ (In fact while proving this you will notice that the fact we are proving is quite intuitive)

Since the two sets have the same size and one contains the other, they must be equal. Hence R(A) = R(B) and C(A) = C(B) implies that A and B are equivalent under row and column permutations.

(Note that we assumed all entries are distinct in the above proof)

This reduced problem can be solved by two approaches:

### 4.4.1 Method 1: Set of Sets Approach (Row and Column sets)

Formally, let

$$R_i = \{a_{i1}, a_{i2}, \ldots, a_{im}\}$$

be the representation of row $i$. We compare:

$$\{R_1, R_2, \ldots, R_n\}_A = \{R_1, R_2, \ldots, R_n\}_B$$

and similarly for columns.

This method is conceptually exact but computationally heavy due to the need to compare complex set-of-sets structures. Set structures can be implemented using balanced binary trees in $\Theta(n \log n)$ time - In `C++` we have the `set` data structure which is implemented as a balanced binary tree. Comparing two sets of size $m$ takes $\Theta(m \log m)$ time, and we have to do this for $n$ rows and $n$ columns, leading to a total time complexity of $\Theta(nm \log n \log m)$.

`C++` implementation:

```
//Matrices are 0-indexed and stored in 2D vectors "first" and "second"
set<set<int>>F;//set of sets for first matrix
for(int i=0;i<n;i++)
{
    set<int>temp(first[i].begin(),first[i].end()); //set of ith row
    F.insert(temp);
}

for(int i=0;i<n;i++)
{
    set<int>temp(second[i].begin(),second[i].end()); //set of ith row
    if (!F.count(temp)) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}
```

### 4.4.2 Method 2: Hashing Based Verification of Rows and Columns

To obtain a more efficient solution, each row is hashed to a pair of values (sum, squaresum), where sum and squaresum denote the sum and sum of squares of elements in the row respectively. This representation is much easier to compare across rows.

Assuming the hash is sufficiently collision-resistant, we compare the sets of row hashes for both matrices. This can be implemented efficiently using `unordered_set` in `C++`, which provides expected $\Theta(1)$ insertion and lookup time.

For each row $i$, we compute:

$$h(R_i) = \left( \sum_{j=1}^{m} a_{ij}, \sum_{j=1}^{m} a_{ij}^2 \right)$$

We define a custom hash for a pair $(x, y)$ as:

$$H(x, y) = x \cdot 2^{32} + y$$

where $x$ and $y$ are the computed row statistics. This choice of pair hash is popular for 32 bit integers, as it combines the two values into a single 64-bit integer guaranteeing unique hashes.

We insert the hashes of all rows of matrix $A$ into an unordered set and then check if the hashes of rows of matrix $B$ are present in this set. If any hash from $B$ is not found in the set from $A$, we can conclude that the matrices are not equivalent under row permutations.

Similarly for columns, we compute the column hashes and compare them in the same manner.

This method has an expected time complexity of $\Theta(nm)$ due to the linear time required to compute the hashes and the expected constant time for hash set operations.

This method involved hashing so the correctness of the algorithm is itself a random variable – but given the constraint that the matrix entries are from {1,2,. . . ,nm}, it seems highly unlikely that this will fail. In fact, it seems that this may be a unique hash in this case, but i am yet to find a proof for general (n,m).

`C++` implementation:

```
auto pair_hash=[](int x,int y){
    return ((x)<<32)+y;
};

unordered_set<long long>s;
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
    {
        int x=first[i][j];
        v1+=x;
        v2+=x*x;
    }
    s.insert(pair_hash(v1,v2));
}
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
    {
        int x=second[i][j];
        v1+=x;
        v2+=x*x;
```

```
    }
    if(!s.count(pair_hash(v1,v2))) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}
```

## 4.5 A Sliding Window Problem

**The Problem** Two integer arrays $x$ and $y$, each of size $n$, are given along with an integer $k$ ($1 \leq k \leq n$). For every contiguous subarray (window) of size $k$, determine whether the corresponding windows in $x$ and $y$ contain the same multiset of elements at every position.

More precisely, for every index (1-based) $i$ such that $1 \leq i \leq n - k + 1$, we compare the multisets:

$$\{x_i, x_{i+1}, \ldots, x_{i+k-1}\} \quad \text{and} \quad \{y_i, y_{i+1}, \ldots, y_{i+k-1}\}$$

and check whether they are identical for all $i$.

### Rolling Hash Method

To obtain an efficient solution, we maintain two rolling hashes for each window:

$$S = \sum a_i, \quad Q = \sum a_i^2$$

These values can be updated in $\Theta(1)$ time when the window slides by removing the outgoing element and adding the incoming element.

For each position $i$, we compute:

$$(S_x(i), Q_x(i)) \quad \text{and} \quad (S_y(i), Q_y(i))$$

and check whether:

$$(S_x(i), Q_x(i)) = (S_y(i), Q_y(i))$$

If the condition holds for all $i$, we conclude that every corresponding window has identical multiset structure.

### C++ Implementation

```
long long sumX=0,sumY=0;
long long sumX2=0,sumY2=0;

for(int i=0;i<k;i++)
{
    sumX+=x[i];
    sumY+=y[i];
    sumX2+=x[i]*x[i];
    sumY2+=y[i]*y[i];
}

bool ok=1;
for(int i=k;i<n;i++)
{
```

```
    if(!((sumX==sumY)&&(sumX2==sumY2)))
    {
        ok=0;
        break;
    }

    sumX-=x[i-k];
    sumX+=x[i];
    sumX2-=x[i-k]*x[i-k];
    sumX2+=x[i]*x[i];

    sumY-=y[i-k];
    sumY+=y[i];
    sumY2-=y[i-k]*y[i-k];
    sumY2+=y[i]*y[i];
}
cout<<(ok?"YES":"NO");
```

This method is very easy to implement and runs in $\Theta(n)$ time, which is efficient for this problem. What was especially nice about this method is that we can maintain the hash values in constant time as the window slides, which is what I meant by using a **rolling hash**. However, it relies on the assumption that the hash values are unique for different multisets. We can improve this (if needed) by doing a second round of hashing with a different hash function like polynomial hashing.

## 4.6 Problems

**1** For hashes of form $H(x) = (ax+b) \bmod c$, why is it preferred to have c as a prime number? *Hint: Think of the arithmetic progression (a very common occurence in data) with common difference d – what happens when d is not coprime with c?*

**2**

# Chapter 5

# Data Structures

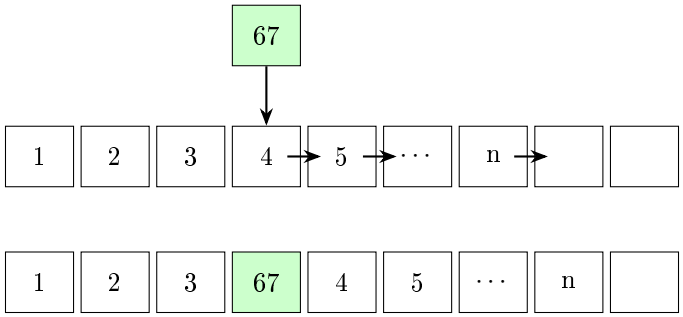
## 5.1 Skip Lists

### Why not just use Arrays?

Suppose we store the following array in contiguous memory:


Suppose we wish to insert the element 67 between 3 and 4.

Since arrays occupy contiguous memory, every element beginning from 4 must be shifted one position to the right in order to create space for the new element.

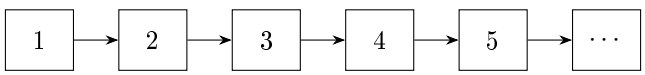



Insertion into an array requires shifting $\Theta(n)$ elements in the worst as well as average case, so a single insertion is $\Theta(n)$ time complexity.

### How are Linked Lists any better?

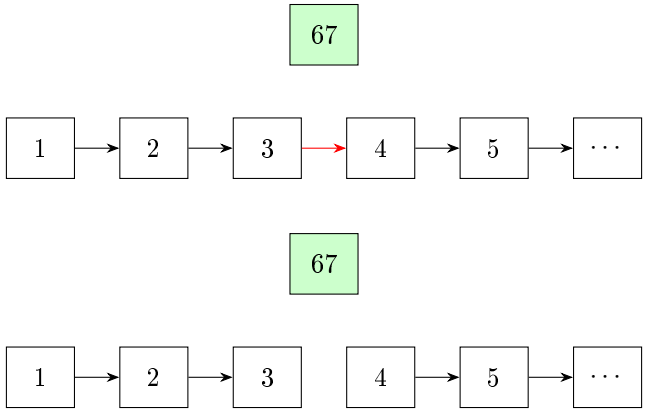
Unlike arrays, linked lists do not store elements contiguously in memory. Instead, each element stores `{Value,Next}`, where `Next` is a pointer to the next element.

So a linked list containing the elements $1, 2, 3, 4, 5, \ldots, n$ may be represented as

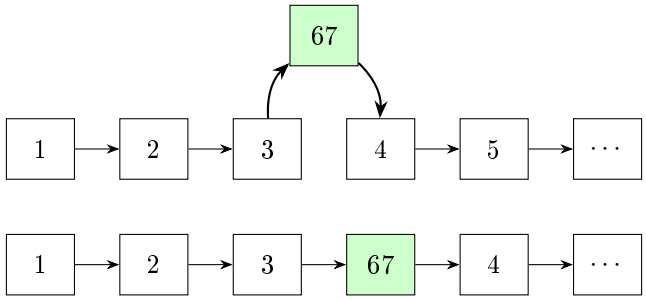



To insert an element 67 between say 3 and 4, we simply create a new node containing 67 and link it between 3 and 4.

**Step 1: Remove the old connection**




**Step 2: Create new connections**




**Done!**

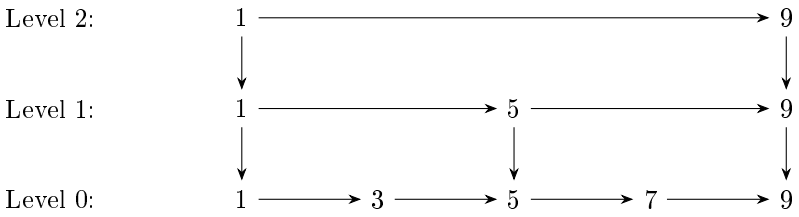
We used a constant number of pointer operations to insert an element into the list. Thus insertion is $\Theta(1)$ time complexity.

## Why Skip Lists?

Although insertion is $\Theta(1)$ time complexity, searching for an element in linked lists is $\Theta(n)$ time complexity because we have to traverse the whole list. Even if the linked list is "sorted", we can not binary search since that requires random access in constant time, which a linked list can not provide.

So the idea of skip lists is to bypass that limitation by adding “express lanes” to a linked list. Instead of traversing every element one-by-one, we maintain additional higher-level lists that skip[1] over many elements at once.

Consider the following skip list –




Higher levels contain fewer elements and allow us to jump across large portions of the list quickly.

[1] Hence the name!

### 5.1.1 The Structure

The presence of an element in a level is determined by a random choice. An element occurs in level $i$ with probability $2^{-i}$. This means every element must occur in level 0.

Consequently, Pr(node reaches level $k$) $= \frac{1}{2^k}$. Thus, the expected number of nodes in level $k$ is $\frac{n}{2^k}$.

| Level | Expected Number of Nodes |
|---|---|
| 0 | $n$ |
| 1 | $n/2$ |
| 2 | $n/4$ |
| 3 | $n/8$ |
| $\vdots$ | $\vdots$ |

### 5.1.2 Expected Height

Let $H$ be the height of a skip list with $n$ independent nodes, where each node is promoted to the next level with probability 1/2. Then:

$$\Pr(H \geq h) = 1 - (1 - 2^{-h})^n$$

We use the tail-sum formula

$$\mathbb{E}[H] = \sum_{h=1}^{\infty} \Pr(H \geq h) = \sum_{h=1}^{\infty} \left(1 - (1 - 2^{-h})^n\right)$$

**Step 1: Split the expectation** Let $h_o = \lfloor \log_2 n \rfloor$

$$\mathbb{E}[H] = \sum_{h \leq h_0} \Pr(H \geq h) + \sum_{h > h_0} \Pr(H \geq h) = S_1 + S_2$$

**Step 2: Bounding $S_1$**

$$\text{Let } Pr(H \geq h) = f(h) = 1 - (1 - 2^{-h})^n \implies f(h_o) \leq f(h) \leq f(0) \; \forall \, 0 \leq h \leq h_o$$

$$\implies \sum_{h=1}^{h_o} f(h_o) \leq \sum_{h=1}^{h_o} f(h) \leq \sum_{h=1}^{h_o} f(0)$$

Since $f(0) = 1$, and $f(h_o) \geq 1 - (1 - \frac{1}{n})^n \geq 1 - \frac{1}{e}$ we get:

$$h_o(1 - \frac{1}{e}) \leq S_1 \leq h_o$$

**Step 3: Bounding $S_2$** Since $(1 - x)^n \geq 1 - nx$,

$$1 - (1 - 2^{-h})^n \leq n \cdot 2^{-h} \implies \sum_{h > h_0} n \cdot 2^{-h} \geq S_2$$

Since LHS of that inequality evaluates to $\approx 2$, we can guarantee $S_2 \leq 3$

**Step 4: Combining bounds**

$$(1 - e^{-1})h_0 + 3 \leq \mathbb{E}[H] \leq h_0 + 3$$

Since $h_0 \approx \log_2 n$, we get $\mathbb{E}[H] = \Theta(\log n)$

**Geometric Decline Theorem**

Recall that using the markov inequality, we proved that the height of the recursion tree for `QuickSelect` is at most $\Theta(\log n)$

Using the same technique, we can show that if a sequence of nonnegative random variables satisfies $\frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq p$ $(0 < p < 1)$ and $T$ denotes the first index for which $X_T \leq 1$, then

$$\mathbb{E}[T] \leq \Theta(\log(X_0))$$

**Applying the Theorem**

Let $X_i$ denote the number of nodes present at level $i$ of the skip list. Since every node is promoted independently with probability 1/2,

$$\mathbb{E}[X_i] = n\left(\frac{1}{2}\right)^i \implies \frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} = \frac{1}{2}$$

The height of the skip list is precisely the highest level containing at least one node. Equivalently, if $H = \min\{i : X_i \leq 1\}$ then the height is H.

Applying the Geometric-Decline theorem with $p = \frac{1}{2}$ gives

$$\mathbb{E}[H] \leq \Theta(\log n)$$

**Why did we do the first method at all?**

In the second method we did not prove a lower bound! Although even in further analyses we won't need the lower bound, it is good to have established that it is exactly $\Theta(\log n)$. Also, we revised the "splitting" technique for finding the complexity of an expression, which is a useful tool in general.

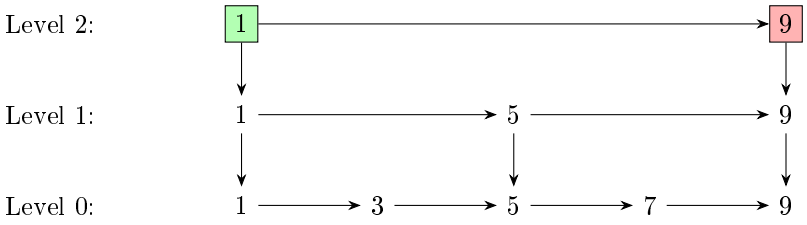
### 5.1.3 Search

Suppose we wish to search for the element 7.

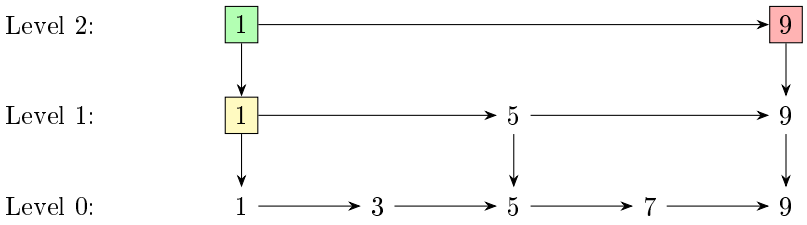
We begin at the top left level and move right while the next element is $\leq 7$. Once that element is strictly greater than 7, we move down to the next level. Repeat until we reach the bottom level.

```
contains(x)
{
    current = top-left node
    while current exists
    {

        while current.right exists and is <= x
            current = current.right

        if current.value == x
            return FOUND

        current = current.down
    }

    return NOT FOUND
}
```

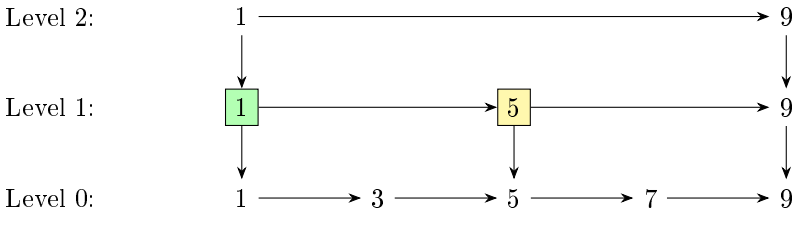
**Step 1: Start at top, check right**




Since on the right we have an element strictly greater than 7, we intend to move down.

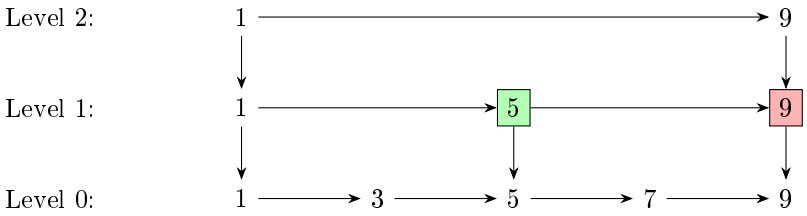



**Step 2: Moved down, now check right**

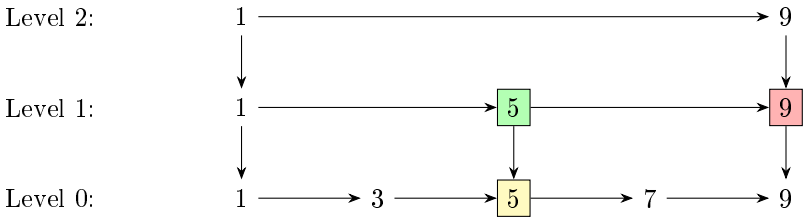



Since $5 \leq 7$, we intend to move right.

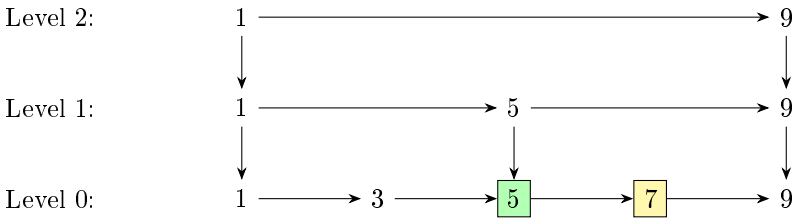
**Step 3: Moved right, check right**



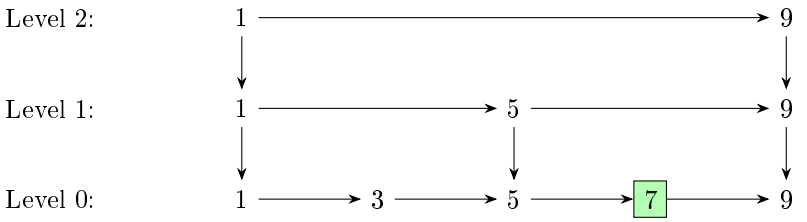

Since $9 > 7$, we intend to move down.




**Step 4: Moved down, check right**




**Step 5: Moved right, found the target!**




We move right to 7 and locate the desired element.

### 5.1.4 Insertion

One may wonder why skip lists use randomization at all. Could we not deterministically place

- every second element in level 1,
- every fourth element in level 2,
- every eighth element in level 3,
- and so on?

While this would indeed support logarithmic search complexity, insertions and deletions would become expensive. For example, inserting a new element near the beginning of the list changes which elements are the second, fourth, eighth, ... elements of the list.

Consequently, many levels may require rebuilding. Randomization avoids this problem by assigning heights independently to each node. Thus, updates are $\Theta(height) = \Theta(\log n)$ while the structure stays balanced in expectation.

The insertion operation in a skip list closely mirrors the search procedure. In fact, insertion is essentially a search followed by a series of pointer modifications. To insert a value $x$, we first perform the standard search starting from the topmost level. At each level, we move right until we either find $x$ or encounter the first node with a key greater than $x$ (or reach the end of the list).

If the key $x$ is found during this traversal, then we do nothing. Otherwise, the search naturally terminates at the position where $x$ should logically appear in the bottom level. This position is determined by the first "blocking" node whose key is greater than $x$, or by reaching a null pointer at the end of the list.

Once this position is identified, insertion proceeds as follows. We insert $x$ immediately to the left of the blocking node at level 0. After placing it in the bottom layer, we decide randomly (typically with probability 1/2) whether the node should also appear in higher levels. If it is promoted, we move upward level by level, and at each level we again insert it in the same relative horizontal position, maintaining alignment with the structure discovered during the search phase.

Thus, the update process reuses the search path and only modifies pointers locally along that path. This is what ensures that skip list insertion remains logarithmic in expected time.

## 5.2 Treaps

Treaps are randomized balanced binary search trees. They combine two ideas:

1. The *binary search tree* property, which allows efficient searching.
2. The *heap* property, which is used to maintain balance.

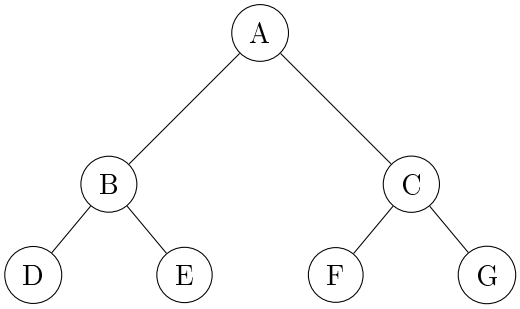
In a simple binary search tree, the insertion order matters. Certain orders can make the height $\Theta(n)$ while others can make it $\Theta(\log n)$. But if we average over all insertion orders, the height is $\Theta(\log n)$. So it seems to avoid adversarial inputs we should randomise the insertion order and the tree will be balanced in expectation! But the whole point of the data structure is we can insert elements at any time so we can not really manipulate the input order.

This brings us to the idea of assigning each element an index (called a priority) just before insertion and then whenever an element is inserted using the standard binary search tree way, we modify the existing tree to pretend as if it was inserted in increasing order of the priorities. To achieve this, we define an operation called *rotation*.

A *rotation* preserves the binary search property while "changing" the order of insertion. Thus we ensure a random insertion order and the treap remains balanced in expectation. This is a relatively simple implementation as compared to Red-Black Trees which stay balanced using rotations as well as recolorings.

### 5.2.1 Binary Trees

A binary tree is a rooted tree in which every node has at most two children, called the left child and the right child. For implementation purposes, the `struct Node` has three pieces of data – its own value, the pointer to its left child, and the pointer to its right child. If the child does not exist, the pointer is set to `nullptr`. We may show an empty node if there is no child for clarity in some figures.



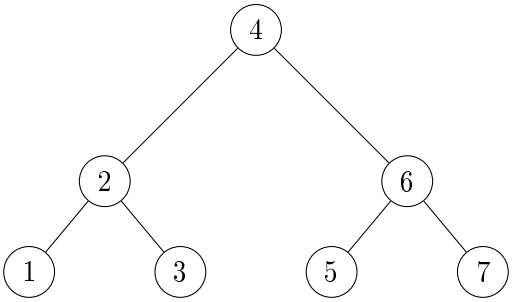

The topmost node is called the **root**. A node with no children is called a **leaf**. Every node together with all of its descendants forms a **subtree**.

### 5.2.2 Tree Traversal

A traversal specifies the order in which nodes are visited. The most important traversal for our purposes is the **In-Order Traversal**.

```
IOT(x)
{
    if (x is null)
        return

    IOT(x.left)
    print(x.value)
    IOT(x.right)
}

IOT(root) \\starting the recursive traversal from the root
```

**Example**



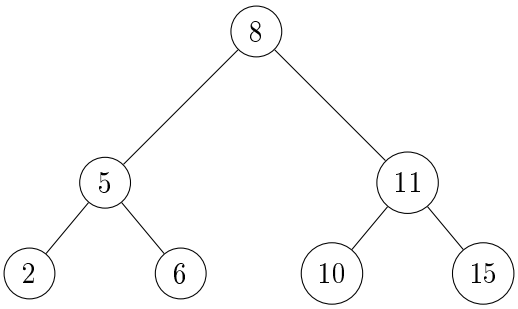

Convince yourself that the output will be `1 2 3 4 5 6 7`. In-Order Traversal is most commonly seen in `Depth-First Search`, popularly known as `DFS`.

### 5.2.3 Binary Search Trees

The value stored at each node is referred to as its **key**. A binary search tree (BST) is a binary tree in which every non-leaf node satisfies

- all keys in the left subtree are smaller than the node's key,

- all keys in the right subtree are larger than the node's key.



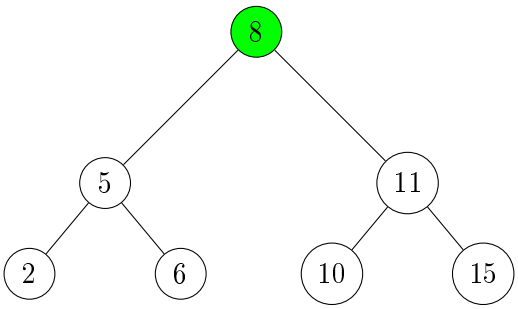

With some thought it can be proved that an In-Order Traversal of a BST lists all keys in sorted order.

### Search

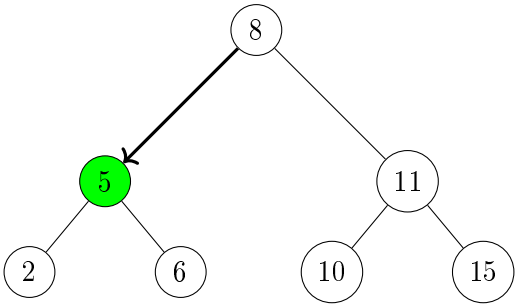
Searching for a key proceeds exactly as in binary search. At each node we compare the desired key with the current key and move either left or right. If the tree is balanced, the tree is the same as the recursion tree of a normal binary search on a sorted array ...

```
Search(key,node)
{
    if (node is null)
        return false

    if(key < node.value)
        return Search(key,node.left)
    if(node.value == key)
        return true
    if(key > node.value)
        return Search(key,node.right)
}

Search(x,root) \\Search the key x, starting from the root
```

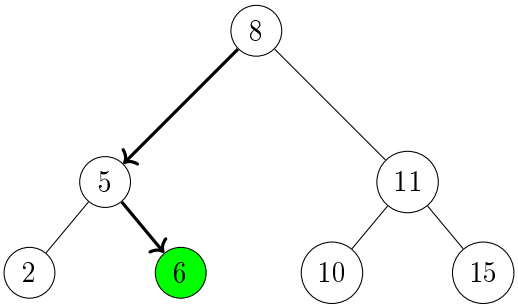
Let us say we are searching for 6 in the tree we saw earlier.




Since 8 is greater than 6, we move left.




Since 5 is less than 6, we move right.



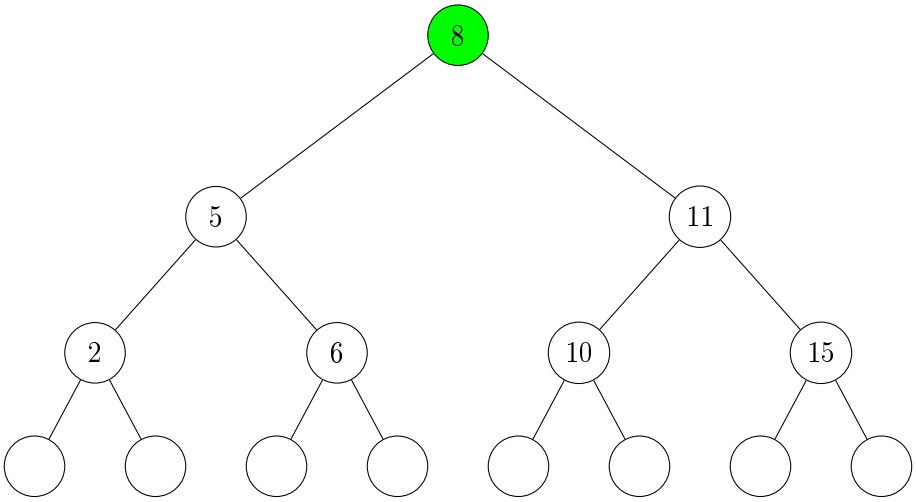

We found it! If we had instead been looking for 7, we would have taken the same path and ended up at the right child of leaf 6 (which is of course, an empty node) so we would have concluded that 7 is not in the tree.

### Insertion

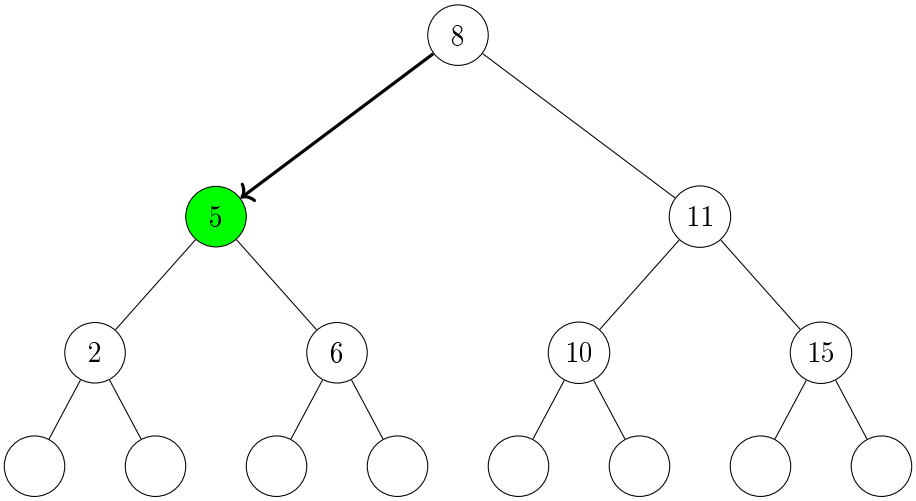
Insertion is similar to binary search. At each node we compare the key to be inserted with the current key and move either left or right. If we find the key in the tree, we do nothing. If we reach an empty node we insert the key there.

```
Insert(key,node)
{
    if (node is null)
    {
        node = new Node(value=key, left=null, right=null)
        return
    }

    if(key < node.value)
        Insert(key,node.left)

    if(node.value == key)
        return

    if(key > node.value)
        Insert(key,node.right)
}

Insert(x,root) \\Insert the key x, recurse starting from the root
```

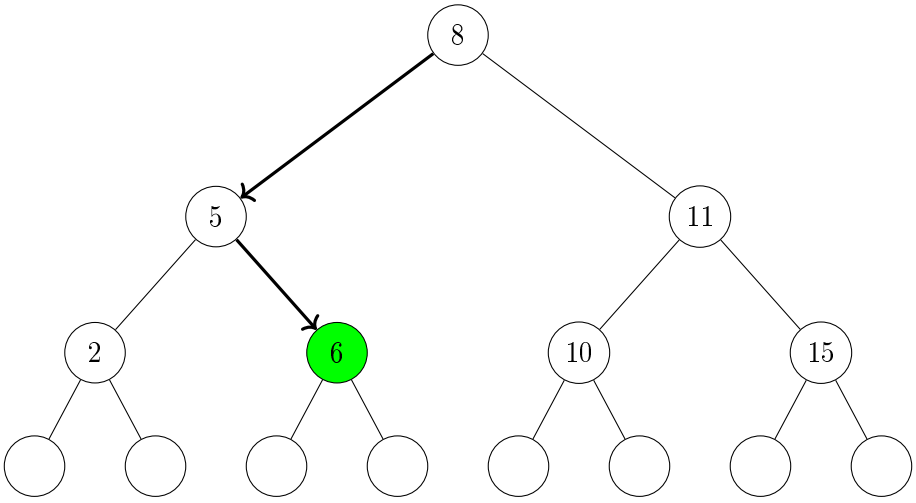
Let us say we want to insert 7 in the tree we saw earlier.



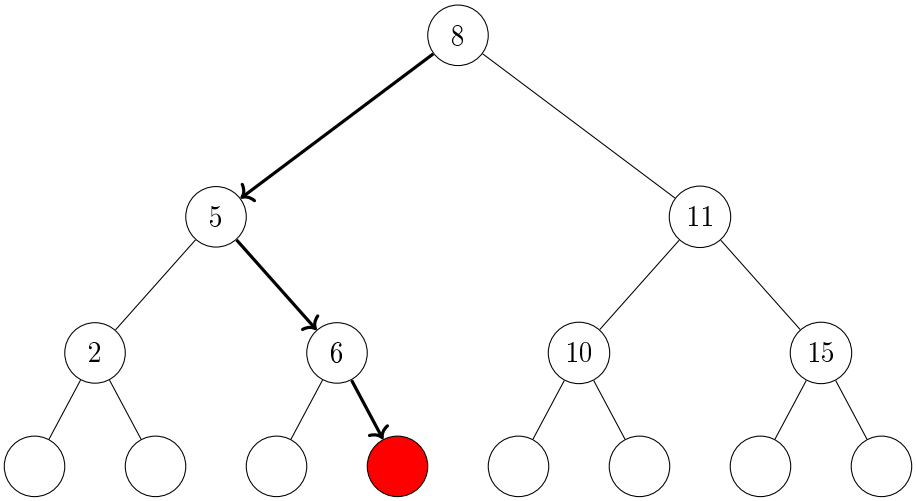

Since 8 is greater than 7, we move left.



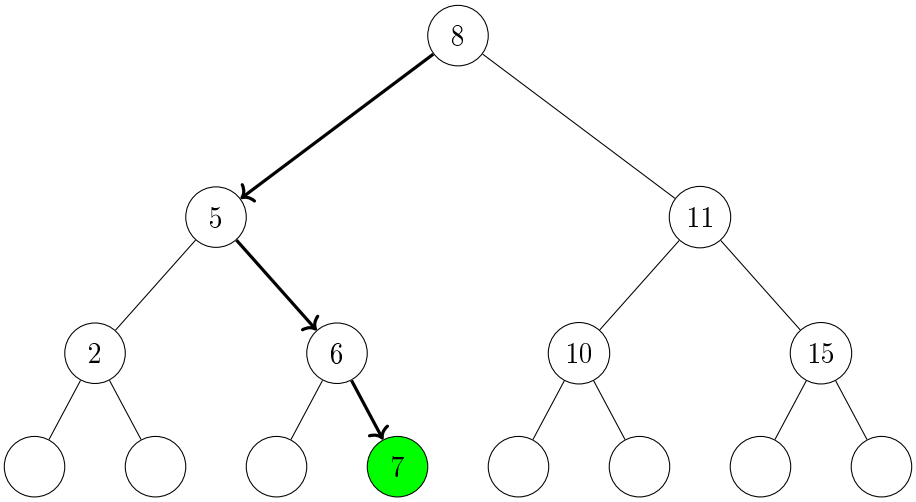

Since 5 is less than 7, we move right.



Since 6 is less than 7, we move right.

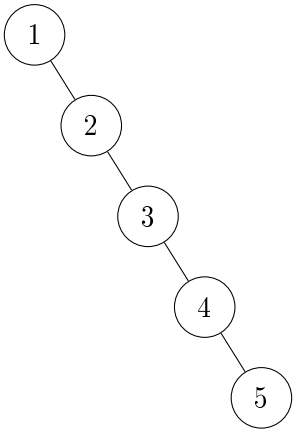



Since the node is empty, we create a new node and put 7 in it.




### 5.2.4 Balancing

The efficiency of a BST depends on its height. If the tree has height $h$, then search, insertion and deletion requires worst-case $\Theta(h)$ time. Ideally we would like $h = \Theta(\log n)$ where $n$ is the number of stored keys. Unfortunately, a BST can become highly unbalanced. For example, inserting the keys $1, 2, 3, 4, \ldots n$ in this order produces a tree with height $n$.



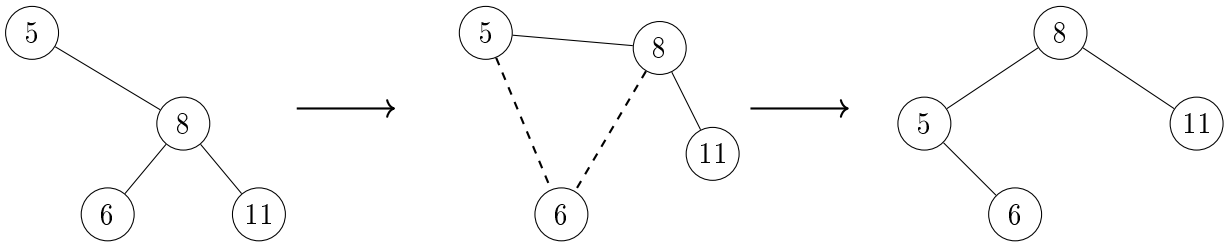

**Example with n = 5**

This tree is essentially a linked list. Its height is $n$ and consequently all operations require $\Theta(n)$ time. To avoid this degeneration we need a mechanism for changing the shape of a tree while preserving the BST property.

### 5.2.5 Rotations

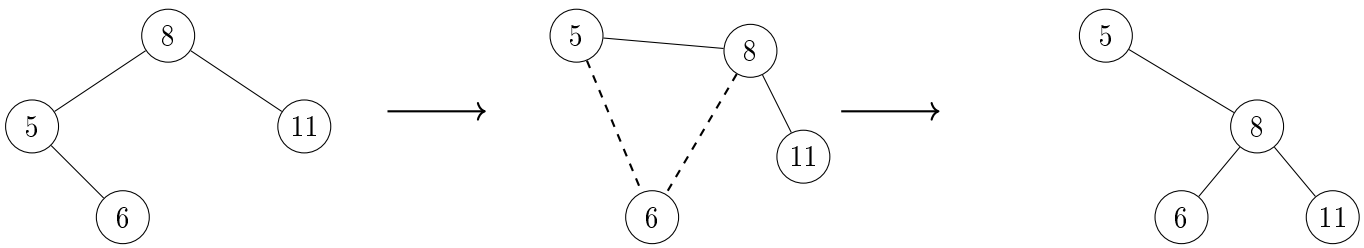
These are operations that transform a tree into another tree with the same inorder traversal.

**Left Rotation**




Before Rotation After

**Right Rotation**




Before Rotation After

### 5.2.6 Treaps

A treap augments every key with a randomly generated priority.

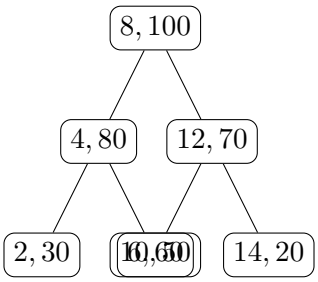
Each node stores

$$(\text{key}, \text{priority}).$$

A treap simultaneously satisfies:

1. The BST property with respect to keys.
2. The heap property with respect to priorities.

For example,




The keys satisfy the BST ordering, while every parent has a larger priority than its children.

The priorities are chosen independently at random when nodes are inserted.

Because the priorities are random, the resulting tree shape behaves like a random BST and is therefore unlikely to become highly unbalanced.

### 5.2.7 Insertion

To insert a new key:

1. Insert it as in a standard BST.
2. Assign a random priority.
3. While the heap property is violated, perform rotations to move the new node upward.

Thus insertion uses rotations to restore the heap property while preserving the BST property.

### 5.2.8 Deletion

To delete a node:

1. Repeatedly rotate the node downward until it becomes a leaf.
2. Remove the leaf.

Again, rotations preserve the BST structure throughout the process.

### 5.2.9 Performance

The priorities induce a random ordering on the nodes.

Consequently, a treap has the same distribution as a random binary search tree.

A classical result states that the expected height of a treap containing $n$ keys is

$$\Theta(\log n).$$

Therefore,

$$\begin{aligned} \text{Search} &= \Theta(\log n), \\ \text{Insertion} &= \Theta(\log n), \\ \text{Deletion} &= \Theta(\log n), \end{aligned}$$

in expectation.

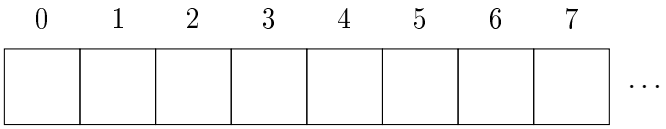
Treaps thus provide a remarkably simple randomized alternative to deterministic balanced trees such as AVL trees and red-black trees.

## 5.3 Hash Tables

The need arose for a data structure to store a set of elements with efficient search, insertion and deletion operations – so we turn to hash tables!

### 5.3.1 The Structure

We begin with an empty array of size $m$. Since this is a plain old normal array of size $m$, we can access and modify any random element in constant time...

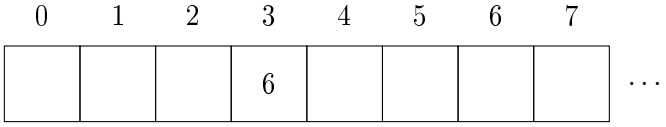



### 5.3.2 Insertion

We add an element $x$ by computing $i = \mathbf{H}(x)$. For example, suppose $\mathbf{H}(6) = 3$. So we place 6 at index 3.

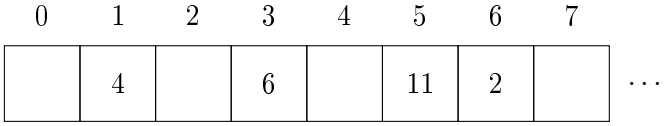
```
Insert(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x present
            return
    }
    add x to end of Linked_List[i]
}
```

**First insertion**



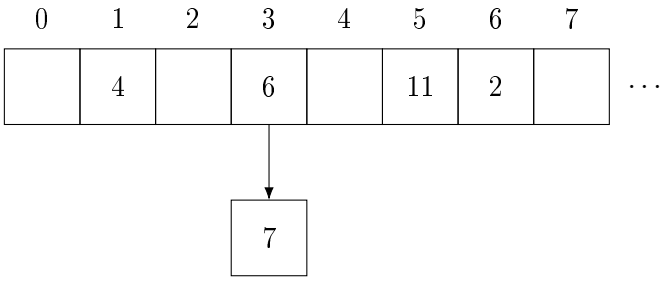

Similarly, we continue inserting elements in the same way as long as no two elements collide at the same index.

**More insertions**

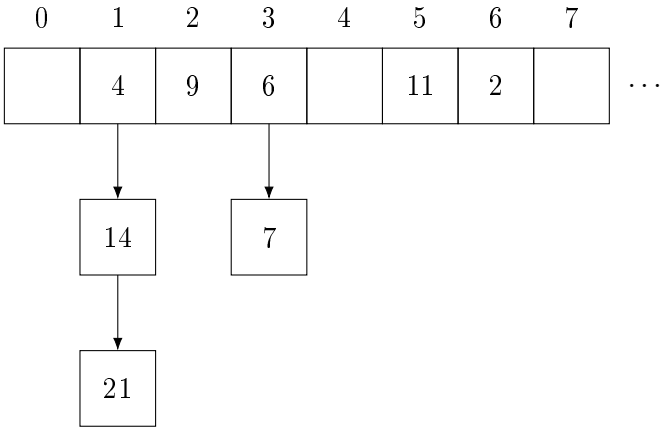



At this point, every index contains at most one element.

**First *collision*** Now we attempt to insert 7. Suppose $\mathbf{H}(7) = 3$. We try to place 7 at index 3, but we find 6 already present. So we extend that position into a linked list.




**More collisions** As more elements are inserted, collisions may occur at multiple indices, forming multiple linked lists.




Thus, each index i stores a linked list of all elements inserted till now who hash to i. Let the total number of elements inserted be $n$ and the number of indices be $m$. Since on average each index has $\frac{n}{m}$ elements, the expected time of insertion is $\Theta(1 + \frac{n}{m})$.

### 5.3.3 Search

```
Search(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x found
            return true
    }
    return false
}
```

Since the expected number of elements per index is $\frac{n}{m}$, the expected time of search is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

### 5.3.4 Deletion

```
Delete(x)
{
    i = H(x)
    traverse Linked_List[i]
    if x found
        remove x from Linked_List[i]
}
```

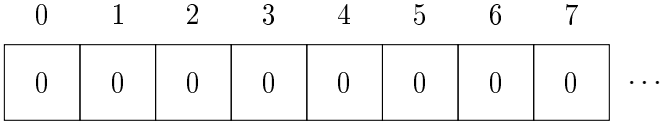
The expected time of deletion is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

## 5.4 Bloom Filters

Suppose we wish to store a set of elements and support the operations `insert(x)` and `contains(x)`. A straightforward solution is to store all inserted elements in an `unordered_set`. This gives expected $\Theta(1)$ insertion and lookup time, but requires memory proportional to the number of stored elements. When the number of elements becomes extremely large, memory itself may become the primary constraint. Bloom filters can achieve this tradeoff.

### 5.4.1 The Structure

Instead of storing the elements themselves, we store only a hash of the set. We begin with a bit array of size $m$ initialised to all zeros.




We then choose $k$ hash functions

$$(H_1, H_2, \ldots, H_k)$$

each mapping elements to the set of indices $\{0, 1, \ldots, m-1\}$.

### 5.4.2 Insertion

To insert an element $x$, we compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and set all corresponding bits to 1. For example, if $k = 3$ and the hash functions are $H_1(x) = 3,\ H_2(x) = 7,\ H_3(x) = 10$ then we set positions $3, 7$, and 10 of the bit array to 1. If while inserting we see that a bit is already 1, then we do nothing.

This operation is $\Theta(k)$ time.

### 5.4.3 Search

To determine whether an element $x$ is present, we again compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and inspect the corresponding bits.

- If at least one of these bits is 0, then $x$ was certainly never inserted.
- If all of these bits are 1, then we report that $x$ is present.

The second conclusion is of course subject to false positives! It is possible that the required bits were set earlier by completely different elements.

This operation is $\Theta(k)$ time.

### 5.4.4 Utility of Bloom Filters

The most important property of Bloom filters is that false negatives are impossible. If the filter reports that an element is absent, then that element was definitely never inserted. The only possible error is reporting that an element is present when it is actually absent.

Interestingly, Bloom filters (as the name suggests) are used as a filter. This means that incoming queries will go through the bloom filter – if the element is absent, we can trust the result. Otherwise we can perform a more expensive lookup in the database. So it is useful to reduce the time taken to process a membership query.

It can be shown[2] that the error probability of a Bloom filter with optimal k is approximately $2^{-\frac{m \ln 2}{n}}$. For this to be small, we need $m$ to be large and $n$ to be small. But one might question, wasn't the whole point of the bloom filter to reduce the memory usage?

Let me clarify this with a practical example – lets say we have a database of URLs. Each URL takes 100 bytes to store. Now let us say we have $10^9$ URLs – and the deterministic `set` structure has a constant factor of about 10 per node in memory. So overall that would require $10^{12}$ bytes of memory which is about 1 TB. In comparison, a Bloom filter of size $m = 10 \cdot n = 10^{10}$ bits would require only about 12.5 GB of memory.

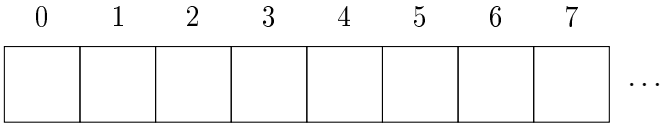
[2] See Problems!

## 5.5 Cuckoo Hashing

Recall that in ordinary hash tables, collisions are handled using linked lists. Cuckoo hashing takes a different approach. Instead of allowing multiple elements to occupy the same location, each element is given multiple possible locations and is stored in exactly one of them.

### 5.5.1 The Structure

We maintain an array of size $m$ together with two hash functions $(H_1, H_2)$.




### 5.5.2 Insertion

Suppose we wish to insert an element $x$. Choose either of the hash functions $H_1$ or $H_2$ at random. Let this chosen hash be applied to $x$ to obtain an index $H(x)$. If the array is empty at that index, we are done. Otherwise, let the occupant be $y$. We evict[3] $y$ and place $x$ in its position. The displaced element $y$ must now be moved to its other possible location. If that position is occupied, another element is evicted, and the process continues.

We repeat this process until we either find an empty slot or find a cycle.

#### Cycles and Rehashing

In case we enter a cycle and insertion cannot be completed, choose new hash functions and rebuild the table from scratch.

Note that overall, as long as we keep $n/m$ low, it can be shown that the expected time to insert an element is $\Theta(1)$ time similar to hash tables.

### 5.5.3 Search

To determine whether an element $x$ is present, we simply check the two possible locations $H_1(x)$ and $H_2(x)$. If $x$ is found at either location, we report that it is present. Thus every lookup requires at most two array accesses, giving $\Theta(1)$ search time.

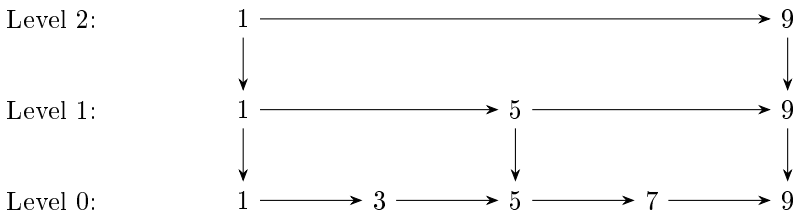
[3] This behaviour resembles the cuckoo bird which lays its eggs in another bird's nest and kicks out the occupying eggs. Hence the name ...

## 5.6 Problems

**1** **(a)** Prove that the expected search time in a skip list is $\Theta(\log n)$. *Hint: downward moves are bounded by height, which is expected* $\Theta(\log n)$*.*

**(b)** Deduce similarly that insertion takes expected $\Theta(\log n)$ time.

**(c)** Try inserting 8 into the following skip list using the procedure we described earlier:




**(d)** Can you think of a $\Theta(\log n)$ deletion strategy for skip lists?

**2** Consider n,m,k to be number of inserted elements, the size of bit array and the number of hash functions respectively in a Bloom filter. Assume the hash functions are independent.

**(a)** Show that the probability of a particular bit in the array being set is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)$ and thus the probability of a false positive is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)^{k}$

**(b)** Using suitable large m, n approximations, show that the optimal number of hash functions is $k=\frac{m}{n}\ln 2$

**(c)** Put the above results together to show that the probability of a false positive is approximately $2^{-\frac{m\ln 2}{n}}$.

# Chapter 6

# Number-Theoretic Algorithms

## 6.1 Introduction

Prime numbers have fascinated mathematicians for many years. They appear deceptively simple to define – a prime is a positive integer greater than one whose only divisors are 1 and itself. Yet many of the deepest questions in mathematics revolve around the distribution and properties of primes.

Beyond their theoretical significance, prime numbers occupy a central position in modern computing. Public-key cryptographic systems such as `RSA` rely on arithmetic involving very large primes, often hundreds or thousands of bits long. As a result, computers routinely face questions such as

- Is a given integer prime?
- How can a large prime be generated efficiently?
- If an integer is composite, can its factors be found quickly?

At first glance, these problems appear daunting. Consider an integer with several hundred decimal digits. Testing divisibility by every smaller integer is clearly infeasible, and even more sophisticated deterministic approaches may require substantial computational effort.

Surprisingly, randomness provides an elegant alternative. Instead of attempting to prove directly that a number is prime, randomized algorithms often search for evidence that it is composite. A carefully chosen random experiment can reveal such evidence with high probability.

In this chapter, we will not be treating multiplication and division as constant time operations. Since these algorithms frequently work with numbers of the order $10^100$ or even larger, we will assume that multiplication and division are $\Theta(\log^2 n)$ time, since a number $n$ has $\Theta(\log n)$ digits.

## 6.2 Fermat Test

The simplest algorithm to test the primality of an integer $n$ is to check for divisibility by every integer from 2 up to $\sqrt{n}$. Using the fact that divison takes $\Theta(\log^2 n)$, the whole algorithm runs in $\Theta(\sqrt{n}\log^2 n)$ time. Any algorithm we come up with should be at least as efficient as this!

The central result underlying our first algorithm is Fermat's Little Theorem. Let the set of primes be denoted by $\mathbb{P}$ and the set $\{0,1,\dots\}$ by $\mathbb{N}$.

### 6.2.1 Fermat's Little Theorem

$$p \in \mathbb{P}, a \in \mathbb{N} \implies a^p \equiv a \pmod{p}$$

A useful equivalent form is obtained if $p \nmid a$, in which case we can divide both sides by $a$ to get

$$a^{p-1} \equiv 1 \pmod{p}$$

This observation immediately suggests a test for primality. Given an integer $n$, choose some value $a$, compute $a^{n-1} \bmod n$, and check whether the congruence holds. If the congruence fails, then $n$ cannot be prime.

### 6.2.2 The Algorithm

```
exp(a, x, n) // computes a^x modulo n in O(log x)
{
    if x == 0
        return 1
    y = exp(a, x/2, n)
    if x even
        return (y*y) mod n
    else
        return (a*y*y) mod n
}

PrimalityTest(n) // for n>=4
{
    if(n even)
        return "composite"

    choose a random a in {2, ..., n-2}

    /*We avoid a=1 and a=n-1 since they trivially satisfy
    the congruence even when n is composite */

    if exp(a, n-1, n) != 1
        return "composite"
    else
        return "probably prime"
}
```

Note the rather beautiful technique used in the function[1] `exp(a, x, n)`. Another way of looking at it is that we find the binary expansion of $x$ and compute the powers of $a$ corresponding to the bits of $x$ using repeated squaring.

### 6.2.3 Error Probability

The natural question is –

"How often can a composite number fool the test?"

[1] This is sometimes called the binary exponentiation algorithm.

For many composite integers the probability is quite small. Unfortunately, there exists a remarkable family of numbers that pass Fermat's test for *every* admissible choice of $a$.

These numbers are known as Carmichael[2] numbers, and they expose a fundamental weakness of the algorithm.

## 6.3 Miller–Rabin Test

### 6.3.1 A Key Property

Let $p$ be an odd prime and let d such that

$$p - 1 = 2^s d, \qquad d \text{ is odd}$$

$$\text{Define } x_i = a^{2^i d} \pmod{p}, i \in \mathbb{N} \iff x_0 = a^d, \quad x_1 = a^{2d}, \quad x_2 = a^{4d}, \quad \ldots, \quad x_s = a^{2^s d}$$

By Fermat's Little Theorem, we have $x_s = a^{p-1} \equiv 1 \pmod{p}$.

Clearly, each term is the square of the previous one modulo $p$. That is,

$$x_{i+1} \equiv x_i^2 \pmod{p}$$

A simple fact about prime moduli –

$$p \in \mathbb{P},\ x \in \mathbb{N},\ x^2 \equiv 1 \pmod{p} \implies x \equiv 1 \pmod{p} \text{ or } x \equiv -1 \pmod{p}.$$

So either $x_0 = 1 \pmod{p}$ or there is some index $i \geq 0$ such that $x_i = -1 \pmod{p}$.

[2]The smallest Carmichael number is 561. It has been proven that there are infinitely many Carmichael numbers.

### 6.3.2 The Algorithm

```
exp(a, x, n)
{
    if x == 0
        return 1
    y = exp(a, x/2, n)
    if x even
        return (y*y) mod n
    else
        return (a*y*y) mod n
}

isPrime(n) // for n>=4
{
    if(n even)
        return "composite"

    s=0
    d=n-1
    while d is even
        increment s by 1, divide d by 2

    //here we have decomposed n-1 into 2^s and d, where d is odd

    Choose a random a in {2, ..., n-2}
    x = exp(a, d, n)

    if x == 1 or x == n-1
        return "probably prime"

    loop i=1 to s-1
    {
        x = (x*x) mod n
        if x == n-1
            return "probably prime"
        if x == 1
        {
            return "composite"
            /*since hitting 1 mod n early means we have found a
            non-trivial square root of 1 mod n, so n is composite*/
        }
    }
    return "composite"
}
```

### 6.3.3 Error Probability

It can be shown that if $n$ is composite, then the probability that the Miller–Rabin test returns "probably prime" is at most 1/4.

The proof[3] that the Miller–Rabin test has error probability at most 1/4 is quite technical and relies on some non-trivial number theory and group theory. Let $T = \{2, \ldots, n-2\}$ , the set of all possible $a$'s. The key and final step is to show that one can partition T into disjoint subsets of size at least 4 such that each subset contains at most one false positive. Thus,

$$|F| \leq |T|/4 \implies \frac{|F|}{|T|} \leq \frac{1}{4}$$

### 6.3.4 Performance

Since there are $\Theta(\log n)$ multiplications involved in binary exponentiation, and each multiplication takes $\Theta(\log^2 n)$ time, the overall total time complexity is $\Theta(\log^3 n)$.

Repeating the test $k$ times will take $\Theta(k \log^3 n)$ time. A reasonable choice of $k$ (e.g., $k = 34$) makes the error probability at most $10^{-20}$.

## 6.4 Pollard's Rho Algorithm

**The Problem** – Given a composite integer $n$, find a non-trivial[4] factor of $n$.

**The Solution** – We can create a pseudo-random sequence of integers $x_0, x_1, \ldots, x_i, \ldots$ such that $x_i \equiv x_{i-1}^2 + c \pmod{n}$ for some $c$.

If at any point $\gcd(x_i - x_j, n)$ is not 1 or $n$, we can take that to be a factor of $n$ by definition[5] of the gcd!

Why do we use the weird sequence $x_i \equiv x_{i-1}^2 + c \pmod{n}$? Why not generate a sequence of random integers and do the same thing? The reason behind using a deterministic sequence is that in this iterative process, we can use Floyd's cycle-finding algorithm to detect collision which takes time linear in the cycle size. In the other case, we would have to store and check with all previous values of $x_j$ for every new $x_i$ generated. That is quadratic in the length of the cycle! Secondly, choosing $x_0$ and $c$ randomly acts as a seed for the pseudo-random process. The sequence is thus overall truly random for our analysis purposes yet is compatible with Floyd's algorithm.

[3] In case you are interested, see https://codeforces.com/blog/entry/98102

[4] In this case, that would be a factor other than 1 and $n$

[5] Conventionally, gcd(0,n) is taken to be n.

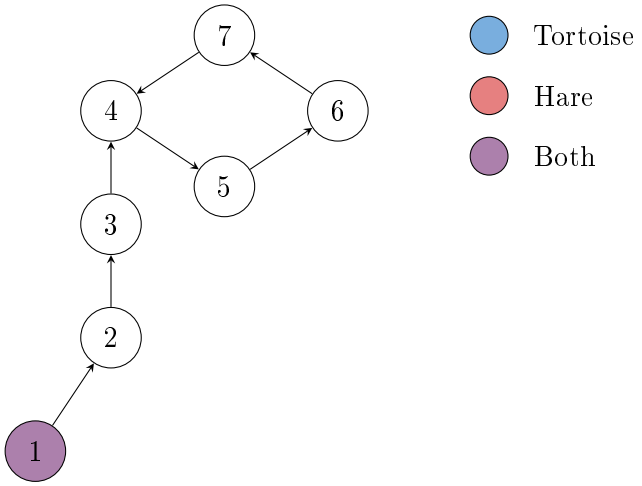
### 6.4.1 Floyd's Cycle-Detection Algorithm

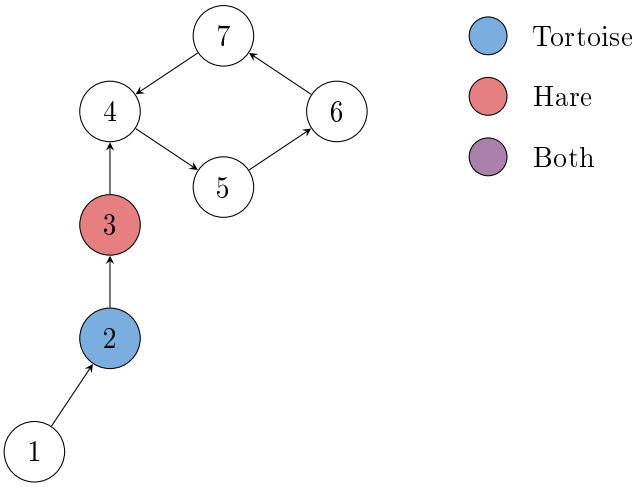
```
FloydCycleDetection(f,x)
{
    tortoise = f(x)
    hare = f(f(x))

    while tortoise != hare:
    {
        tortoise = f(tortoise)
        hare = f(f(hare))
    }

    return
}
```

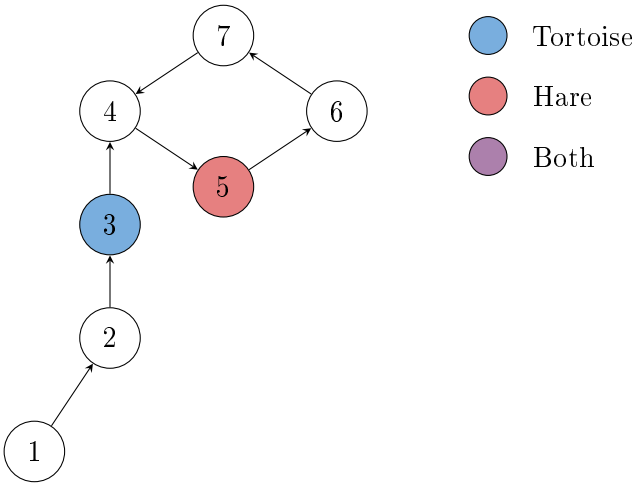
#### Example

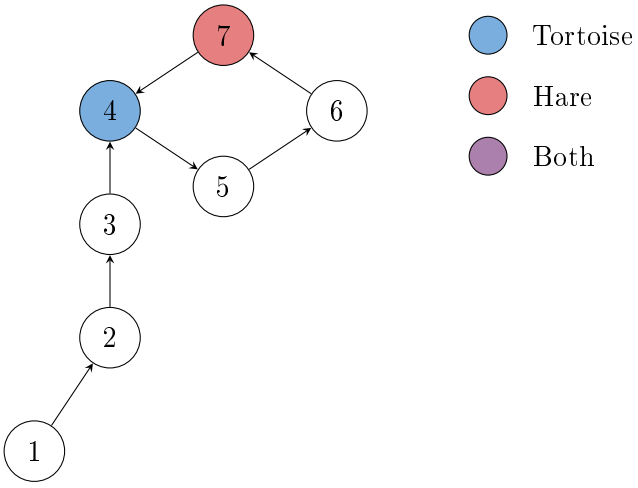
We can represent the sequence with a directed graph as follows – there is an edge between each $x$ and $f(x)$ or equivalently each $x_i$ and $x_{i+1}$. The starting node is $x_0$ where both the hare and tortoise start. It can be proven that Floyd's algorithm works with some effort, but the point of this example is to give a visual intuition rather than write a rigorous proof. Also, I like colorful graphs!

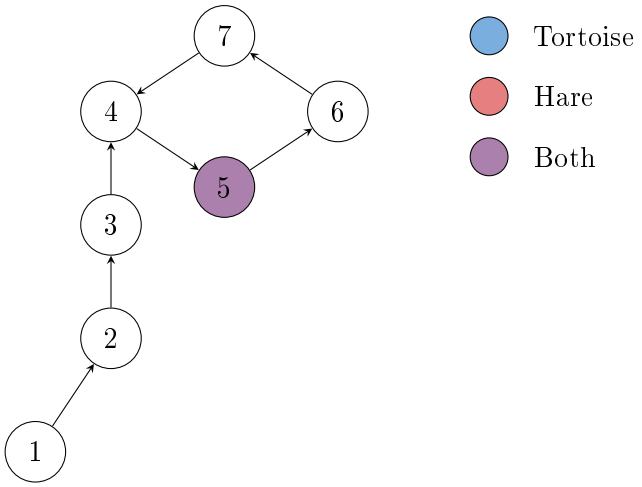
The shape of the graph is deliberately chosen to be like the greek letter $\rho$, which is where the name `Pollard's Rho` comes from. It is the general shape of the sequence which the algorithm considers, because the output space of $f(x)$ is finite so it must go into a cycle by pigeonhole principle.




7
4
6
5
3
2
1
Tortoise
Hare
Both


7
4
6
5
3
2
1
Tortoise
Hare
Both


7
4
6
5
3
2
1
Tortoise
Hare
Both




And thus when both coincide, we can report we have found the cycle! Try this on a cycle with a different (perhaps odd) number of nodes to really convince yourself that it works.

### 6.4.2 The Algorithm

```
f(t)
{
    return (t*t + c) mod n
}

PollardRho(n)
{
    if n is even:
        return 2

    choose random c in [1, n-1]
    choose random x in [0, n-1]

    y = x
    d = 1

    while d == 1:
    {
        x = f(x)
        y = f(f(y))

        d = gcd(|x - y|, n)
    }

    if d == n:
        restart algorithm with new x, c

    return d
}
```

### 6.4.3 Performance

#### Collision Model

Our sequence $x_0, x_1, x_2, \ldots$ is generated by iterating a polynomial function modulo $n$. Since our choice of $x_0$ and $c$ is random,the sequence can be treated as independent samples from a uniform distribution over $\{0, 1, \ldots, n-1\}$, until repetition occurs. And we are looking for a hidden number $p$, the modulo over which the sequence repeats. Since $p$ divides $n$, the sequence when taken modulo p will thus act like independent samples from a uniform distribution over $\{0, 1, \ldots, p-1\}$.

Equivalently it can be said that the distribution Uniform(0,n-1) modulo p is equivalent to the distribution Uniform(0,p-1).

This naturally leads us to the birthday paradox[6] model, where we are interested in the expected time until a collision occurs in a sequence of random samples from a finite set.

Suppose we define the collision step $S$ as the smallest index such that a repetition occurs,

$$\iff S = \min\{j \mid x_i = x_j \text{ for some } i < j\}$$

#### Probability of No Collision

Let $P_k$ denote the probability that the first $k$ samples are all distinct. This is equivalent to $Pr(S > k)$. Then using the fact that the samples are uniform and independent, we can express $P_k$ as

$$P_k = Pr(S > k) = \prod_{i=0}^{k-1} \left(1 - \frac{i}{p}\right)$$

#### Expected Collision Step

The expected collision time can be expressed as

$$\mathbb{E}[S] = \sum_{k=0}^{\infty} Pr(S > k) = \sum_{k=0}^{\infty} P_k = 1 + \left(1 - \frac{1}{p}\right) + \sum_{k=2}^{\infty} P_k = 2 - \frac{1}{p} + \sum_{k=2}^{\infty} P_k$$

We will use $2 - \frac{1}{p} \leq 2$ and the inequality $1 - x \leq e^{-x}$ to bound this.

$$P_k = \prod_{i=0}^{k-1} \left(1 - \frac{i}{p}\right) \leq \prod_{i=0}^{k-1} e^{-i/p} = e^{-\sum_{i=0}^{k-1} i/p} = e^{-k(k-1)/(2p)}$$

$$\implies \mathbb{E}[T] \leq 2 + \sum_{k=2}^{\infty} e^{-k(k-1)/(2p)}$$

For $k \geq 2$,

$$k(k-1) \geq \frac{k^2}{2} \implies \frac{k(k-1)}{2p} \geq \frac{k^2}{4p} \implies e^{-k(k-1)/(2p)} \leq e^{-k^2/(4p)}$$

$$\implies \mathbb{E}[S] \leq 2 + \sum_{k=2}^{\infty} e^{-k^2/(4p)}$$

[6] The probability that in a group of 23 people we find two with the same birthday is approximately 1/2. This is of course, not a true *paradox* but simply a surprising result from an intuition standpoint.

Since the function $f(x) = e^{-x^2/(4p)}$ is monotonically decreasing on $[0, \infty)$, we can compare the sum with an integral!

$$\sum_{k=2}^{\infty} e^{-k^2/(4p)} \leq \int_0^{\infty} e^{-x^2/(4p)}\, dx.$$

Using the substitution $x = 2\sqrt{p}\, t,\ dx = 2\sqrt{p}\, dt$,

$$\implies \int_0^{\infty} e^{-x^2/(4p)}\, dx = 2\sqrt{p} \int_0^{\infty} e^{-t^2}\, dt = 2\sqrt{p} \cdot \frac{\sqrt{\pi}}{2} = \sqrt{\pi p}.$$

$$\implies \mathbb{E}[S] \leq 2 + \sqrt{\pi p} \implies \mathbb{E}[S] \leq \Theta(\sqrt{p})$$

### Collision Detection

Once a collision occurs modulo $p$, say

$$x_i \equiv x_j \pmod{p}, \quad i \neq j,$$

we know $p$ must divide $x_i - x_j$. But how do we check that if we don't know $p$? Knowing we want to find a $p$ which divides $x_i - x_j$ and $n$, the logical step is to find the $\gcd(x_i - x_j, n)$, because if that is 1 then p is definitely[7] 1. And as soon as $\gcd(x_i - x_j, n)$ becomes not 1 (and not n), we can take that as the divisor. We know all p for which the collision happened are definitely[8] smaller than this number so we could not have done better!

Due to Floyd's cycle detection algorithm, we don't need to keep track of all pairs of indices $i, j$. We can simply keep track of the pairs $(x_i, x_{2i})$ for $i = 1, 2, \ldots$ and check for collisions between these pairs. This is what the algorithm does by maintaining two pointers $x$ and $y$ that move at different speeds.

### Cost Per Step

Each step of the algorithm performs a constant number of modular multiplications and one greatest common divisor computation. Using the Euclidean algorithm, $\gcd(a, b)$ can be computed in at most $\Theta(\log^3 max(a, b)))$ time.

Hence each iteration costs at most $\Theta(\log^3 n)$ time since n is the largest number we are working with for all the involved operations.

### Total Expected Complexity

Since cost per step is at most $\Theta(\log^3 n)$, and the expected number of steps is at most $\Theta(\sqrt{p})$, the total expected complexity is at most $\Theta(\sqrt{p} \log^3 n)$ for a particular prime $p$.

Let $n = \prod_{i=1}^{k} p_i^{e_i}$ where $p_1 < p_2 < \cdots < p_k$ are the prime factors of $n$. Let $T_i$ denote the random time until a collision occurs modulo $p_i$, and let $T$ be the running time of `Pollard's Rho`, i.e.

$$T = \min_{1 \leq i \leq k} T_i$$

Since $T \leq T_i$ for every $i$, taking expectations gives

$$\mathbb{E}[T] \leq \mathbb{E}[T_i] \quad \forall i \in \{1, 2, \ldots, k\} \implies \mathbb{E}[T] \leq \min_{1 \leq i \leq k} \mathbb{E}[T_i]$$

[7] This is concluded using the lemma – all common divisors of a and b divide the gcd(a,b)

[8] By definition of the gcd, it must be the *greatest* common divisor!

$$\implies \mathbb{E}[T] \leq \Theta(\sqrt{p_1}\log^3 n), \quad \text{where } p_1 \text{ is the smallest prime factor of } n$$

$$p_1 <= \sqrt{n} \implies \mathbb{E}[T] \leq \Theta(\sqrt{p}_1 \log^3 n) \leq \Theta(n^{\frac{1}{4}} \log^3 n)$$

### What did we achieve?

We can find a factor of $n$ in expected time $\Theta(n^{\frac{1}{4}} \log^3 n)$, a vast improvement over the naive $\Theta(n^{\frac{1}{2}} \log^2 n)$ method.

To factorise a large number n fully, we can use the Pollard-Rho algorithm now! Since every factor we find is at least 2, after dividing n by 2 and repeating the factor-finding process, the overall procedure terminates in at most $\Theta(\log_2 n)$ steps. Thus we can factorise a number n (no matter how large), in expected time at most $\Theta(n^{\frac{1}{4}} \log^4 n)$.

Note that we have used a lot of worst case upper bounds in the analysis. The average performance in practice is much better than the complexity suggests!

## 6.5 Problems

**1** During RSA encryption, one computes a secret integer $x = \phi(n)$. A valid public exponent $e$ must satisfy

$$\gcd(e, x) = 1.$$

Since we do not want to reveal $x$ to the public, we can not take $e$ to be something deterministic like $x - 1$ because an adversary can crack our encryption if he gets to know the algorithm. One possible strategy is to repeatedly choose $e$ uniformly at random from $\{1, 2, \ldots, x\}$ until a suitable value is found.

**(a)** Let

$$x = \prod_{i=1}^{k} p_i^{e_i}$$

be the prime factorization of $x$. Show that the probability that a randomly chosen $e \in \{1, 2, \ldots, x\}$ satisfies $\gcd(e, x) = 1$ is

$$\frac{\varphi(x)}{x} = \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right).$$

**(b)** Show that if x can be any number from 1 to $n$ where $n$ is very large, the probability of the randomly chosen e being correct tends to $\frac{6}{\pi^2}$ as n becomes larger.
*Hint: Use* $\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$

# Chapter 7

# Probabilistic Method

## 7.1 Dominating Set

**The Problem** Given a graph:

$$G = (V, E)$$

find a smallest possible subset:

$$D \subseteq V$$

such that every vertex in the graph is either:

- contained in $D$, or
- adjacent to some vertex in $D$.

Such a set is called a *dominating set*.

Intuitively, we may think of the vertices in $D$ as "control centers" or "facilities" such that every location in the graph is either itself a facility or directly connected to one.

Unfortunately, finding the minimum dominating set is NP-hard, so we want to find a polynomial time algorithm that gives us a reasonably small[1] dominating set.

### A Randomised Solution

Suppose the graph has minimum degree $d$. We now consider the following extremely simple randomised strategy:

> Choose $k$ vertices randomly out of the n vertices (uniformly among the $\binom{n}{k}$ possible choices). If the chosen set forms a dominating set, output it. Otherwise, try again!

At first glance, this algorithm seems dumb. However, it turns out that if we choose $k$ large enough, the probability of failure becomes small. The key question thus becomes:

> What is the probability of failure as a function of $k$, $n$, and $d$?

We now analyse this probability.

[1] We care more about the asymptotic behaviour of the size of set produced by the algorithm and less about which polynomial time complexity the algorithm has.

### Probability That a Vertex Remains Undominated

Fix a vertex $v \in V$. Since the graph has minimum degree $d$, $N[v]$ contains at least $d+1$ vertices. For $v$ to remain undominated, none of the selected $k$ vertices may lie inside $N[v]$

When choosing vertices uniformly at random, the probability that a single chosen vertex does *not* belong to $N[v]$ is at most:

$$1 - \frac{d+1}{n}$$

$$\implies \Pr[v \text{ remains undominated}] \leq \left(1 - \frac{d+1}{n}\right)^k$$

Using the inequality[2] $1 - x \leq e^{-x}$ for all $x \in \mathbb{R}$, we obtain:

$$\Pr[v \text{ remains undominated}] \leq e^{-k(d+1)/n}$$

### Bounding the Failure Probability

The algorithm fails if at least one vertex remains undominated.

Applying the union bound[3]

$$\Pr[\text{failure}] \leq n \cdot e^{-k(d+1)/n}$$

We would like this probability to be strictly less than 1. Therefore we require:

$$ne^{-k(d+1)/n} < 1$$

$$\implies \ln n - \frac{k(d+1)}{n} < 0$$

$$\implies k > \frac{n \ln n}{d+1}$$

Thus, choosing:

$$k = \frac{cn \ln n}{d+1}$$

for any constant $c > 1$ gives $\Pr[\text{failure}] \leq n^{1-c}$ which tends to 0 as $n \to \infty$.

### What Does This Actually Mean?

The important point is not merely that the algorithm succeeds with high probability.

Rather, we have shown something stronger –

> There exists a dominating set[4] of size roughly:
>
> $$\frac{n \log n}{d}$$

Indeed, if no such dominating set existed, our algorithm could never possibly succeed.

Thus a simple probabilistic argument simultaneously:

- proves the existence of a reasonably small dominating set,
- and gives a randomised algorithm capable of finding one.

[2] Refer Appendix B.1

[3] Refer Appendix A.3

[4] This method has a limitation - if $d < c \ln n$, then the size of the dominating set produced by the algorithm is basically the whole set of vertices, making it useless.

### Amplifying the Success Probability

Even if a single run fails, we may simply repeat the algorithm independently.

If the failure probability of one run is $\boldsymbol{p} = \frac{1}{n^{c-1}}$, then after $\boldsymbol{t}$ independent repetitions, the probability that *every* run fails equals $\boldsymbol{p^t}$ which decreases exponentially fast.

So the tradeoff is choosing a larger $c$ has a smaller expected number of repetitions $(\frac{n^{c-1}}{n^{c-1}-1})$[5] but a larger dominating set, while choosing a smaller $c$ has a larger expected number of repetitions but a smaller dominating set.

## 7.2 Bipartite Subgraph

**Problem statement.** Given an undirected graph $G = (V, E)$ with $|E| = m$, we wish to find a bipartition of $V$ that maximizes the number of edges between vertices of different partitions. Equivalently, we want a 2-coloring of the vertices that maximizes the number of bichromatic edges. We aim to show that we can always guarantee at least $m/2$ edges in the cut.

### 7.2.1 Expected Number of Bichromatic Edges

Let X be the number of bichromatic edges under a random 2-coloring of the vertices. Each edge has a probability $\frac{1}{2}$ of being bichromatic, so define the indicator variable $I_i$ as 1 when the edge numbered $i$ is bichromatic and 0 otherwise. We can conclude that for a random vertex 2-coloring, $\Pr(I_i = 1)=\frac{1}{2}$ and $\Pr(I_i = 0)=\frac{1}{2}$, so $\mathbb{E}[I_i] = \frac{1}{2}$. Since $X = \sum_{i=1}^{m} I_i$, we have:

$$\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{m} I_i] = \sum_{i=1}^{m} \mathbb{E}[I_i] = \frac{m}{2}$$

### A Weak Lower Bound

Let $X \in \{0, 1, \ldots, m\}$ be the number of bichromatic edges under a random 2-coloring of the vertices. We already have $\mathbb{E}[X] = \frac{m}{2}$.

We derive a crude lower bound on $\Pr\big(X \geq \frac{m}{2}\big)$.

**Case 1:** $m = 2k$ **(even).**

Let $p = \Pr(X \geq k)$. The maximum value of $E(X)$ under these constraints would be achieved when we push all mass in $\{0, 1, \ldots, k-1\}$ to $k-1$, and all mass in $\{k, \ldots, 2k\}$ to $2k$. Then

$$\mathbb{E}[X] \leq (1-p)(k-1) + p(2k).$$

Using $\mathbb{E}[X] = k$,

$$\begin{aligned} k &\leq (1-p)(k-1) + 2kp \\ k &\leq k - 1 + p(k+1) \\ 1 &\leq p(k+1) \\ p &\geq \frac{1}{k+1} = \frac{2}{m+2}. \end{aligned}$$

$$\implies \Pr\Big(X \geq \frac{m}{2}\Big) \geq \frac{2}{m+2}.$$

[5] Since the probability of failure is $p = (n^{c-1} - 1)/n^{c-1}$ and this is a bernoulli trial, the expected number of trials for success is 1/p

**Case 2:** $m = 2k + 1$ **(odd).**

Let $p = \Pr(X \geq k+1)$. The maximum value of $E(X)$ under these constraints would be achieved when all mass in $\{0, 1, \ldots, k\}$ is pushed to $k$, and all mass in $\{k+1, \ldots, 2k+1\}$ is pushed to $2k+1$. Then

$$\mathbb{E}[X] \leq (1-p)k + p(2k+1).$$

Using $\mathbb{E}[X] = k + \frac{1}{2}$,

$$k + \frac{1}{2} \leq k + p(k+1)$$

$$\frac{1}{2} \leq p(k+1)$$

$$p \geq \frac{1}{2(k+1)} = \frac{1}{m+1}.$$

$$\implies \Pr\Big(X \geq \frac{m}{2}\Big) \geq \frac{1}{m+1}.$$

**Conclusion** –

$$\Pr\Big(X \geq \frac{m}{2}\Big) \geq min(\frac{2}{m+2}, \frac{1}{m+1}) = \frac{1}{m+1}.$$

### 7.2.2 A Simple Randomised Algorithm

From the probabilistic analysis, a random coloring satisfies

$$\Pr\left(X \geq \frac{m}{2}\right) \geq \frac{1}{m+1}.$$

Therefore, if we repeatedly sample random colorings independently, the probability that none of $T$ trials succeeds is at most

$$(1 - \frac{1}{m+1})^T.$$

Choosing $T = (m+1)\ln(m+1)$ gives

$$(1 - \frac{1}{m+1})^T \leq e^{-\frac{T}{m+1}} = e^{-\ln(m+1)} = \frac{1}{m+1}$$

Which goes to 0 as $m \to \infty$.

While simple, this method does not exploit structure of intermediate solutions and relies purely on repeated random sampling rather than improvement steps.

### 7.2.3 Derandomisation

Instead of making random choices, we assign vertices one by one smartly, trying to maximise the number of bichromatic edges. Suppose some vertices have already been assigned and the remaining vertices are still unassigned. Let X be the random variable denoting the number of bichromatic edges. Let L and R be the bipartitions.

Denote the expected value of X given the first $i$ assignments by:

$$\mathbb{E}[X \mid i]$$

For the $i+1$-th assignment of vertex $v$, we have two choices:

- place $v$ in $L$,

- place $v$ in $R$.

The current conditional expectation is the average of the expectations obtained from these two choices. Therefore, at least one choice must yield a conditional expectation that is at least as large as the current one.

$$\max\left(\mathbb{E}[X \mid \text{i, v in L}], \mathbb{E}[X \mid \text{i, v in R}]\right) >= \frac{1}{2}\left(\mathbb{E}[X \mid \text{i, v in L}] + \mathbb{E}[X \mid \text{i, v in R}]\right) = \mathbb{E}[X \mid i]$$

We simply assign the next vertex such that it would add the most bichromatic edges in this step – that choice yields a better expectation for the future.

$$\text{Since I have ensured that } \mathbb{E}[X \mid i+1] \geq \mathbb{E}[X \mid i]$$

$$\implies \mathbb{E}[X \mid \text{n assignments}] \geq \mathbb{E}[X \mid \text{0 assignments}]$$

$$\implies X_{final} \geq \frac{m}{2}$$

**Why did that work?**

Initially $\mathbb{E}[X] = \frac{m}{2}$. At every step we choose an assignment that does not decrease the conditional expectation[6]. Hence the conditional expectation never drops below $m/2$!

After all vertices have been assigned, no randomness remains. The conditional expectation is then equal[7] to the actual number of crossing edges produced by the algorithm.

$$\implies X_{final} \geq \frac{m}{2}.$$

Thus we obtain a deterministic algorithm that always constructs a bipartite subgraph containing at least half of the edges.

**Why does this not work for all problems?**

The properties of this particular problem we needed for this to work were –

1. $E[X \mid 1] = m/2$, regardless of which assignment that was – symmetry between the left and right sides.
2. It was obvious which assignment would maximise $E[X \mid i+1]$ given $E[X \mid i]$.

Both these properties do not hold, for example, in the Dominating Set problem. Depending on your first choice (to put in the dominating set or dominated set), you might get a very good dominating set in expectation or a very bad one. An extreme counter example is a star shaped[8] graph, where if you put the central node in dominated set you will get a dominating set of size $n-1$, but if you put it in dominating set you will get a dominating set of size 1. The second property also fails because in further choices it is not clear how to choose which set to put the node in to maximise the conditional expectation.

[6] This is by no means a proof for optimality – it does not maximise the number of bichromatic edges.

[7] If you are feeling tricked somehow, I relate fully! But it is true, we designed this deterministic algorithm and simultaneously proved that it produces at least $m/2$ crossing edges using conditional expectation!

[8] A star shaped graph is a graph with one central node and $n-1$ nodes connected to it and only it.

## 7.3 Array Construction

### The Problem

In this problem, we must construct an array of $4n$ integers in which each of the values $1, 2, \ldots, n$ appears exactly four times, subject to the following condition – for a value $x$, let $p_{x,1} < p_{x,2} < p_{x,3} < p_{x,4}$ be the positions of its four occurrences. Then for every $x$ the three consecutive gaps

$$g_1 = p_{x,2} - p_{x,1}, \qquad g_2 = p_{x,3} - p_{x,2}, \qquad g_3 = p_{x,4} - p_{x,3}$$

must be pairwise distinct. For instance, with $n = 3$ the array $[1, 1, 2, 1, 2, 3, 1, 3, 2, 2, 3, 3]$ works because the gaps are $(1, 2, 3)$ for 1, $(2, 4, 1)$ for 2 and $(2, 3, 1)$ for 3, each consisting of three different numbers.

### The Solution

Consider the naive randomised algorithm of simply generating a random array of $4n$ integers which have 4 of each of $\{1, 2, 3, \ldots, n\}$. We can then check if the condition holds for each of the $n$ values and if not, generate again. This is a simple Las Vegas algorithm but as we will see, pretty effective in generating the required construction quickly.

### Success Probability

Let the four positions of $x$, in increasing order, be $a_1 < a_2 < a_3 < a_4$, and define the gaps

$$g_1 = a_2 - a_1, \qquad g_2 = a_3 - a_2, \qquad g_3 = a_4 - a_3,$$

each a positive integer, with span $s = g_1 + g_2 + g_3 = a_4 - a_1$. Once the gaps are fixed, the subset is determined by the starting position $a_1$ alone, which must satisfy $a_1 \geq 1$ and $a_1 + s = a_4 \leq N$.

There are therefore exactly $N - s$ admissible subsets for each tuple of gaps with $s \leq N - 1$. Also note that the probability of two gaps being equal is the same regardless of which pair of gaps we are referring to (by symmetry).

$$\Pr(g_1 = g_2) = \Pr(g_2 = g_3) = \Pr(g_1 = g_3)$$

#### Two Equal Gaps

When $g_1 = g_2 = g$ and $g_3 = h$, $s = 2g + h$. So the number of such subsets are $N - (2g + h)$. Double summation over all possible $g$ and $h$ will yield all valid subsets and thus the exact probability.

$$\Pr(g_1 = g_2) = \frac{1}{\binom{4n}{4}} \sum_{g=1}^{2n-1} \left( \sum_{h=1}^{4n-2g-1} (4n - 2g - h) \right) = \frac{1}{\binom{4n}{4}} \sum_{g=1}^{2n-1} \binom{4n-2g}{2} = \frac{n(2n-1)(8n-5)}{3 \cdot \binom{4n}{4}}$$

$$\implies \Pr(g_1 = g_2) = \frac{n(2n-1)(8n-5)}{n(2n-1)(4n-1)(4n-3)} = \frac{8n-5}{(4n-1)(4n-3)} \approx \frac{1}{2n} + \Theta(\frac{1}{n^2})$$

**All Equal Gaps**

Here $g_1 = g_2 = g_3 = g$, so $s = 3g$ and the number of subsets is $4n - 3g$.

The summation over g will go from 1 to $G = \left\lfloor \frac{4n-1}{3} \right\rfloor$.

$$\sum_{g=1}^{G}(4n - 3g) = 4nG - \frac{3G(G+1)}{2} = \frac{G\,(8n - 3G - 3)}{2}$$

$$\implies \Pr(g_1 = g_2 = g_3) = \frac{G\,(8n - 3G - 3)}{2\binom{4n}{4}} = \frac{3\,G\,(8n - 3G - 3)}{(2n)(2n-1)(4n-1)(4n-3)}$$

Taking large n approximations and $G \approx \frac{4n}{3}$ gives

$$\Pr(g_1 = g_2 = g_3) \approx \frac{4n\,(8n - 4n)}{(2n)(2n)(4n)(4n)} \approx \frac{1}{4n^2}$$

**Inclusion-Exclusion**

$$\Pr(\text{bad value}) = \Pr((g_1 = g_2) \cup (g_2 = g_3) \cup (g_1 = g_3))$$

$$= \Pr(g_1 = g_2) + \Pr(g_2 = g_3) + \Pr(g_1 = g_3) - 3 \cdot \Pr(g_1 = g_2 = g_3) + \Pr(g_1 = g_2 = g_3)$$

$$= 3\Pr(g_1 = g_2) - 2 \cdot \Pr(g_1 = g_2 = g_3) \approx 3 \cdot (\frac{1}{2n}) - 2 \cdot (\frac{1}{4n^2}) + \Theta(\frac{1}{n^2}) \approx \frac{3}{2n}$$

$$\implies \Pr[\text{bad value}] \approx \frac{3}{2n}$$

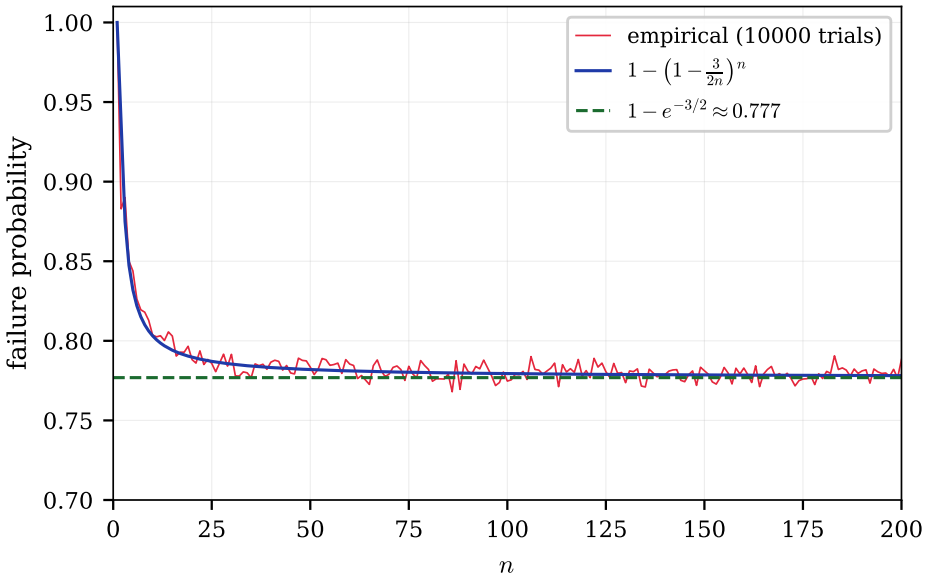
**Overall Success Probability**

The arrangement fails exactly when the gaps of at least one value is bad. The union bound gives $n \cdot \frac{3}{2n} = \frac{3}{2}$ as the upper bound on overall failure which is useless. To find anything useful, we need to know something about how they are dependent on each other. We will skip the exact analysis of this interdependence and notice that the placement of one value affects the gaps of other value by at most 4. Since this won't affect most arrangements, they are almost[9] independent.

$$\Pr[\text{success}] \approx (1 - \Pr(\text{bad value}))^n \approx \left(1 - \frac{3}{2n}\right)^n$$

Since each run takes $\Theta(n)$ time (generation and verification), the overall runtime is also expected $\Theta(n)$ using Wald's equation[10] since we have expected number of runs to be $1/\Pr[\text{success}]$ which is constant hence $\Theta(1)$.

[9] Yes, this is an approximation but the graph of empirical data should convince you how good it is.
[10] Refer Appendix A.8




Interestingly, the method used to generate this graph is also a randomised algorithm – the method of sampling. We used the idea of law of large numbers by saying that the number of failures divided by total number of runs (10000) should be a good approximation to the failure probability. Of course, 10000 was chosen because it was large as compared to the $n$ under consideration.

# Chapter 8

# Online Algorithms

## What is an Online Algorithm?

An online algorithm is an algorithm that processes its input piece-by-piece in a serial fashion, without having the entire input available from the start. In some problems, this doesn't matter – the obvious solution is online since it requires a single pass over the array only. But in some problems, the online requirement makes the problem much harder and forces us to make decisions without knowing the future.

Online decision algorithms have randomness involved in their analysis for non-adversarial[1] inputs. Further, the goal of the algorithm usually is not deterministic optimality, but rather to maximise the probability of success or to achieve a good competitive ratio[2].

## 8.1 Dating

### The Problem

Consider the problem of finding the best partner. Assume that the number of people you will eventually date is fixed, i.e. some known value $n$. After dating a person, you must immediately decide whether to commit to them forever or reject them permanently and continue searching. Figure out a strategy that maximises the probability of ending up with the best partner.[3] We assume that while dating person $i$, we only know how they compare relative to the people we have already dated. So we could sort the partners till now in a total order if needed.

### The Solution

Most course materials, internet reels I have encountered this problem in directly jump to the strategy

> "Reject the first $r$ people, then accept the first person afterwards who is better than everyone seen so far."

However, they often omit the proof of why an optimal strategy must necessarily have this threshold form. We first prove this fact.

[1] They are often called oblivious adversaries

[2] Competitive ratio is a measure of how well an online algorithm performs compared to an optimal offline algorithm that has access to the entire input from the start. The term n-competitive is used when an algorithm achieves at least $\frac{1}{n}$ times the "performance" of an optimal offline algorithm.

[3] This is a different problem from maximising the expected "value" of your partner. Here the objective is specifically to maximise the probability of selecting the single best partner among all $n$ people.

### Optimality Proof

Suppose we are currently considering the $k$-th person.

If the current person is *not* the best among the first $k$ people seen so far, then accepting them can never succeed: This is because there already exists an earlier person who is strictly better, and that earlier person has already been rejected. Hence an optimal strategy should only ever consider accepting people who are the best seen so far.

Now suppose the current person *is* the best among the first $k$ people seen so far. Let $\boldsymbol{q_k}$ be the optimal probability of eventually selecting the globally best person if we reject the $k$-th person and continue optimally afterwards.

**Q] If we instead accept the current person immediately, what is the probability of success?**

A] Since the ordering of the $n$ people is uniformly random, each of the first $k$ positions is equally likely to contain the globally best person. Conditioned on the fact that the current person is the best among the first $k$, the probability that they are actually the globally best person equals $\frac{k}{n}$.

Therefore, accepting gives success probability $\frac{k}{n}$, rejecting gives success probability $q_k$. The optimal decision is to accept iff $\frac{k}{n} \geq q_k$[4].

Now observe that $\frac{k}{n}$ is **increasing** with $k$.

On the other hand, $q_k$ is **decreasing** with $k$. To see this, notice that from time $k$, one valid strategy is:

> "Reject one more person no matter what, and then behave optimally from time $k+1$ onwards."

This strategy achieves success probability exactly $q_{k+1}$. Since $q_k$ is the *optimal* achievable success probability starting from time $k$, we must have

$$\mathbf{q_k \geq q_{k+1}}$$

Since $\frac{k}{n}$ is increasing and $q_k$ is decreasing, the optimal decision changes at most once from rejecting to accepting as $k$ increases. Therefore there exists some threshold $0 \leq r \leq n$ such that

- for $k \leq r$, rejecting is optimal,
- for $k > r$, accepting the first "best-so-far" person is optimal.

Hence every optimal strategy must be a threshold strategy. Note that $r = 0$ corresponds to accepting the first person, and $r = n$ corresponds to rejecting everyone and ending up with no partner. Both of these are valid threshold strategies, though they are not very good ones[5].

[4] This is a common theme in online algorithms - the optimal decision at each step is determined by comparing the expected "value" of actions and choosing the one with maximum value. Here our "value" is the success probability. In game theory this is called "utility" instead.

[5] Though it may be argued that having no partner is the best choice - this is beyond the scope of this text and is left as an exercise for the reader.

### Finding the Optimal Threshold

We now analyse the threshold strategy –

> "Reject the first $r$ people, then accept the first person afterwards who is better than everyone seen so far."

Suppose the globally best person appears at position $k$. For our strategy to succeed,

1. We must have $k > r$
2. Among the first $k-1$ people, the best person must appear within the first $r$ positions.

The first condition ensures that we do not automatically reject the best person. The second condition ensures that no earlier candidate after position $r$ triggers acceptance before the globally best person appears.

$$\Pr[\text{best person appears at position } k] = \frac{1}{n}$$

Further, among the first $k-1$ people, each position is equally likely to contain their maximum.

$$\implies \Pr[\text{maximum among first } k-1 \text{ lies in first } r] = \frac{r}{k-1}$$

$$\implies \Pr[\text{success}] = \sum_{k=r+1}^{n} \frac{1}{n} \cdot \frac{r}{k-1} = \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} = \frac{r}{n} \left(H(n-1) - H(r-1)\right)$$

Using the approximation $H(x) \approx \ln(x)$,

$$\frac{r}{n} \left(H(n-1) - H(r-1)\right) \approx \frac{r}{n} \ln\left(\frac{n-1}{r-1}\right) \approx \frac{r}{n} \ln\left(\frac{n}{r}\right)$$

$$x = \frac{r}{n} \implies P(x) = x \ln\left(\frac{1}{x}\right)$$

$$\implies P'(x) = \ln\left(\frac{1}{x}\right) - 1$$

$$P'(x_o) = 0 \implies \ln\left(\frac{1}{x_o}\right) = 1 \implies x_o = \frac{1}{e}$$

Since $\frac{r_o}{n} = \frac{1}{e} \implies r_o = \frac{n}{e}$, the optimal strategy is –

> "Reject approximately the first $\frac{n}{e}$ people, then accept the first person afterwards who is better than everyone seen so far."

The corresponding success probability is $\frac{1}{e} \ln e = \frac{1}{e}$ which is approximately **36.8**%. Not bad at all!

Strictly speaking, we should also verify that this critical point indeed corresponds to a maximum rather than a minimum. One way is via the second derivative test –

$$P''(x) = -\frac{1}{x} < 0, \ \forall\, x \in \mathbb{R}^{>0}$$

so $P(x)$ is concave and hence the critical point is necessarily a global maximum.

Alternatively, we can avoid calculus entirely by inspecting the behaviour near the boundaries.

$$\because P(x) = x \ln\left(\frac{1}{x}\right)$$

As $x \to 0, P(x) \to 0$ since $x$ decays faster than $\ln(1/x)$ grows.

Similarly, as $x \to 1, P(x) \to 0$ because $\ln(1) = 0$.

Since $P(x)$ is positive for $0 < x < 1$, rises away from 0, and eventually falls back to 0, the unique critical point at $x = \frac{1}{e}$ must correspond to the global maximum.

### Behaviour for Small $n$

We used some limiting n approximations in finding the optimal threshold (I hope you noticed!). In case you were not planning on dating a tending-to-infinite number of people, you might be interested in the optimal strategy for small values of $n$. For finite $n$, the exact success probability for threshold $r$ is

$$\max_{0 \le r \le n} \left( \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} \right)$$

Computationally we can now easily find max for each $n$

| $n$ | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|
| $n/e$ | 0.74 | 1.10 | 1.47 | 1.84 | 2.21 | 2.58 |
| round($n/e$) | 1 | 1 | 1 | 2 | 2 | 3 |
| Actual optimal $r$ | **1** | **1** | **1** | **2** | **2** | **3** |

The corresponding optimal success probabilities are

| $n$ | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|
| Pr[success] | **0.500** | **0.500** | **0.458** | **0.433** | **0.428** | **0.414** |

We observe that the optimal threshold follows the value suggested by $\frac{n}{e}$ even for relatively small values of $n$, and it will get closer and closer as $n$ increases. The success probability is already around 41.4% for $n = 7$, and it will approach the limiting value of $\frac{1}{e} \approx 36.8\%$ as $n$ grows.

## 8.2 Load Balancing

Consider a setting with $m$ identical machines processing incoming tasks. The tasks arrive one after another, and the arrival times are sufficiently close that we may assume they arrive within negligible time of each other.

Each incoming task must immediately be assigned to one of the $m$ machines. Once a decision is made, it cannot be reversed. Thus, the algorithm has no knowledge of future tasks while making its decision.

The tasks assigned to a machine are processed sequentially and hence form a queue at that machine. Suppose the incoming tasks have processing times $T_1, T_2, \ldots, T_n$ where $n$ itself is not known in advance.

The objective is to minimize the *makespan*, the finishing time of the busiest machine, equivalently $\max_{1 \le i \le m} T_i$ where $T_i$ denotes the total processing time assigned to machine $i$. This is a natural objective since the overall processing time is determined by the machine that finishes last[6].

## Greedy Algorithm

A natural strategy is the following greedy algorithm.

```
Assign(Task T)
{
    find machine with min load
    assign T to this machine
}
```

### Performance Analysis

Interestingly, we do not actually have a polynomial time algorithm to find the optimal *offline* solution to this problem.

We will therefore use lower bounds on $OPT$ (the optimal offline makespan) to derive an upper bound on the competitive ratio of the greedy algorithm.

Let $ALG$ denote the makespan produced by the greedy algorithm. Consider the task that finishes last in the greedy schedule. This may or may not be the last *assigned* task. Let its processing time be **p**.

Immediately before assigning this job, suppose the minimum machine load was $L$. By our hypothesis that this machine will be the one that finishes last, the greedy makespan is

$$ALG = L + p$$

Hence the total load already assigned before this final job was at least **mL**. Including the final job, the total processing load is therefore at least $\mathbf{mL + p}$.

By Pigeonhole Principle, at least one machine must have load at least the average load, so –

$$OPT \ge \frac{mL + p}{m} = L + \frac{p}{m}$$

Since any valid schedule must process the job of size $p$,

$$OPT \ge p \implies OPT(1 - \frac{1}{m}) \ge p\left(1 - \frac{1}{m}\right)$$

Adding the two inequalities gives:

$$OPT(2 - \frac{1}{m}) \ge L + p = ALG \implies \boxed{\frac{ALG}{OPT} \le 2 - \frac{1}{m}}$$

Therefore the greedy algorithm is $\left(2 - \frac{1}{m}\right)$-competitive, sometimes written as 2-competitive since the difference is negligible for large $m$.

[6] This concept is similar to the Rate Determining Step (RDS) in the field of Chemical Kinetics

**Tightness of the Bound**

The previous analysis showed that the greedy algorithm is $\left(2 - \frac{1}{m}\right)$-competitive. We now construct an example showing that the analysis cannot, in general, be improved beyond a factor of $\left(2 - \frac{1}{m}\right)$.

Consider $m = 4$ machines and the following sequence of incoming jobs:

$$1, 1, 1, \ldots, 1 \text{ (12 times)}, 4$$

The greedy algorithm will assign the first 12 jobs as follows:

| Machine $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Jobs assigned | 1, 1, 1 | 1, 1, 1 | 1, 1, 1 | 1, 1, 1 |

After processing these 12 jobs, each machine has load 3. The last job of size 4 will be assigned to any machine (say machine 1), resulting in a final makespan of 7. On the other hand, an optimal offline algorithm could have assigned the first 12 jobs as follows:

| Machine $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Jobs assigned | 1, 1, 1, 1 | 1, 1, 1, 1 | 1, 1, 1, 1 | 4 |

After processing these 13 jobs, each machine has load 4. Thus the optimal makespan is 4, and the competitive ratio is $\frac{7}{4}$, which is indeed our bound of $2 - \frac{1}{4}$.

## 8.3 Distinct Elements

We consider a stream of elements

$$a_1, a_2, \ldots, a_m$$

drawn from a the finite set $U$. The goal is to write an online algorithm that estimates the number of distinct elements (denoted by D) in the stream. We have the constraint that both m and D are large (say $2^{100}$) so we can not use $\Theta(D)$ space like using a RBT[7]. In fact, we would really like to use a small amount of space as compared to $D$.

### 8.3.1 Flajolet–Martin Method

We take a hash function $\mathbf{H} : U \to \{0,1\}^N$, where we ensure[8] $2^N \approx m^2$. This hash function is not being used in the traditional sense of reducing memory required to store the element but rather a way to convert an element to a random[9] bitstring. For each element $x$, define $\rho(x)$ as the number of leading zeros in $\mathbf{H}(x)$.

We then maintain $\rho_{max}$ as we go through the input stream. At the end of the stream, our estimate for $D$ is $D = \alpha \cdot 2^{\rho_{max}}$

**Why $\rho_{max}$?**

The statistic max is multiplicity-insensitive, i.e. repeated occurences of an element in the input do not change the value of the max whereas something like "the number of elements with $\rho = 3$" will be skewed by multiple occurences of an element.

[7] This is shorthand for a Red-Black Tree which is used in the implementation of the `set` in `C++`

[8] This choice of N ensures a small collision probability as is required for this algorithm.

[9] As always, we will take pseudo-random as true random for our purposes

**Why $\alpha \cdot 2^{\rho_{max}}$?**

For a random bitstring, $P(\rho(x) \geq k) = 2^{-k}$. For each level $k \geq 0$ and index of a distinct element $e \in \{1, \ldots, D\}$, define the indicator variable

$$I_{k,e} = \begin{cases} 1 & \text{if } \rho(e) \geq k \\ 0 & \text{otherwise} \end{cases}$$

The point of defining the indicator variable was to define

$$D_k = \sum_{z=1}^{D} I_{k,z}$$

so that $D_k$ counts the number of distinct elements whose hash values have at least $k$ leading zeros.

Since a uniformly random hash has at least $k$ leading zeros with probability $2^{-k}$,

$$E[I_{k,z}] = P(\rho(x) \geq k) = 2^{-k}$$

Since the variables $I_{k,1}, I_{k,2}, \ldots, I_{k,D}$ are i.i.d.[10], the Strong Law of Large Numbers gives

$$\frac{D_k}{D} = \frac{1}{D} \sum_{z=1}^{D} (I_{k,z}) \xrightarrow{\text{a.s.}} E[I_{k,z}] = 2^{-k}$$

$$\implies D_k \approx D \cdot 2^{-k}$$

So we expect $\rho_{max}$ to be when $D_{\rho_{max}}$ drops to 1,

$$D_{\rho_{max}} \approx 1 \implies D \cdot 2^{-\rho_{max}} \approx 1 \implies D \approx 2^{\rho_{max}}$$

**What is $\alpha$?**

If we do multiple parallel runs of the algorithm (essentially using different hash functions and maintaining $\rho_{max}$ separately for each) and take simple average over them (given enough number of runs the Strong Law of Large Numbers will hold), then $\alpha = \frac{E[2^{\rho_{max}}]}{D}$, which we can find since we know the distribution of the random variable $\rho_{max}$ with respect to a fixed D.

The cumulative distribution function of $\rho_{max}$ is

$$Pr(\rho_{max} \geq k) = 1 - (1 - 2^{-k})^D$$

The calculus to find $\alpha$ is quite involved so we just skip to stating $\alpha \approx 1.29281$.

**Modification**

The average of the random variable $2^{\rho_{max}}$ takes quite a large number of samples to converge to the mean because it has a long right tail (which means the larger values skew the mean a lot). This can also be observed by the large variance of the random variable $2^{\rho_{max}}$.

Let $R_1, R_2, \ldots, R_m$ be the values obtained from $m$ independent runs and compute the average of $2^{-R_i}$ rather than the usual $2^{R_i}$ or equivalently use the harmonic mean of $2^{R_i}$. The modified estimate is then proportional to harmonic mean of $2^{R_i}$ with $\beta_m$ as constant factor.

$$D = \beta_m \cdot HM(2^{R_i}) = \frac{\beta_m}{Avg(2^{-R_i})}$$

[10] This is shorthand for independent and identically distributed

The random variable $2^{-R_i}$ has a smaller variance hence will help us converge to the correct answer faster.

### 8.3.2 HyperLogLog Method

Instead of designing multiple independent hashes as in Flajolet–Martin, we can achieve the same effect using a single hash function and storing multiple max statistics of the data. Let $\mathbf{H}(x) \in \{0,1\}^N$ exactly as defined before. We now split the hash into two parts,

$$\mathbf{H}(x) = B(x) \,\|\, S(x)$$

where the first $p$ bits $B(x)$ determine a bucket. A particular bucket is defined as all those elements whose hash starts with a particular bitstring. Thus there are $b = 2^p$ different buckets – while reading the input stream, we classify each element into one of these buckets.

Define $\rho(x)$ as the number of leading zeros in the bitstring $S(x)$. For each bucket we maintain its $\rho_{max}$.

What we have done is stored $b$ different i.i.d. max statistics of the input stream and we can take the harmonic to get an estimate for number of elements in each bucket. Intuitively, each bucket receives approximately $\frac{D}{b}$ distinct elements and each bucket behaves like a smaller run of the Flajolet–Martin Method.

As before, HyperLogLog introduces a constant factor $\beta_b$ and estimates number of distinct elements in one bucket using the harmonic mean of $2^{R_i}$ as

$$\beta_b \cdot HM(2^{R_i}) = \frac{\beta_b}{Avg(2^{-R_i})}$$

So D is estimated as $b$ times the above estimate so

$$D = \frac{\beta_b \cdot b}{Avg(2^{-R_i})}$$

Again, finding the value of the constant $\beta_b$ is a bit complicated so we skip to stating

$$\beta_b = \left( m \int_0^\infty \left( \log_2 \left( \frac{2+u}{1+u} \right) \right)^m du \right)^{-1} \approx \frac{0.7213}{1 + \frac{1.079}{b}}$$

**Memory Usage**

Each bucket needs a variable which stores the maximum number of leading zeros observed in its bucket. The maximum number of leading zeros is atmost $N$ since that is the length of the hash. To store the number $N$ we need $\log_2 N$ bits. Since chose $N$ such that $2^N \approx m^2$, we have a space requirement of

$$\log_2(2 \log_2 m) = \Theta(\log(\log(m)))$$

With $b$ buckets, the total memory requirement is

$$\Theta(b \log \log m)$$

In practice $b$ is chosen to be a fixed constant such as $2^{10} = 1024$ so for $m = 2^{100}$, the memory requirement is about $2^{10} \cdot \log_2(2 \log_2(2^{100})) \approx 8000$ bits which is very small.

# Chapter 9

# Heuristic Algorithms

Many computational problems of practical interest have a statement like "Given a graph find a set of vertices of size atmost x satisfying property P."
Now one way to guarantee a solution is to find the minimum sized set of vertices satisfying P, but this is usually not doable in polynomial time.
So we are often forced to settle for a solution that is not necessarily optimal but can be found efficiently. Such algorithms are *heuristic algorithms* – algorithms designed to quickly find solutions that are often good in practice, even if they do not always provide provable optimality guarantees or even any guarantees at all.
Inherently the analysis of heuristic algorithms involve probability and expectation, making them a natural application of the tools we have developed so far.

Notice that the greedy algorithm we saw in the previous chapter was a heuristic algorithm as well!

## 9.1 Dominating Set Revisited

Recall the dominating set problem: given a graph $G = (V, E)$, find a smallest possible subset $D \subseteq V$ such that every vertex in the graph is either contained in $D$ or adjacent to some vertex in $D$.

### A Greedy Heuristic

A very natural approach is the following greedy algorithm:

> Repeatedly select the vertex that dominates the largest number of currently undominated vertices.

More precisely:

1. Initially, all vertices are marked undominated.
2. At each step, choose a vertex whose closed neighbourhood $N[v]$ contains the largest number of undominated vertices.
3. Add this vertex to the dominating set.
4. Mark all vertices in $N[v]$ as dominated.
5. Repeat until every vertex becomes dominated.

At every step we greedily maximise immediate progress.

### Analysing the Greedy Heuristic

Let OPT denote the size of a minimum dominating set.

Suppose that at some stage of the algorithm there remain $r$ undominated vertices.

Since the optimal solution uses only OPT vertices to dominate the entire graph, those same OPT vertices must also collectively dominate these remaining $r$ vertices.

Therefore, by the pigeonhole principle, at least one of those optimal vertices must dominate at least $\frac{r}{\text{OPT}}$ of the currently undominated vertices.

But the greedy algorithm always chooses a vertex dominating as many undominated vertices as possible. Hence the greedy step must dominate at least $\frac{r}{\text{OPT}}$ new vertices.

Thus after one greedy step, the number of remaining undominated vertices becomes at most

$$r - \frac{r}{\text{OPT}} = r\left(1 - \frac{1}{\text{OPT}}\right)$$

If $r_t$ denotes the number of undominated vertices after $t$ greedy steps, then

$$r_{t+1} \leq r_t \left(1 - \frac{1}{\text{OPT}}\right)$$

Since initially $r_0 = n$, we obtain

$$r_t \leq n \left(1 - \frac{1}{\text{OPT}}\right)^t$$

We would like all vertices to become dominated, i.e. $r_t < 1$.

Since

$$r_t \leq n \left(1 - \frac{1}{\text{OPT}}\right)^t$$

it suffices to find the smallest $t$ such that

$$n \left(1 - \frac{1}{\text{OPT}}\right)^t < 1$$

Rearranging:

$$\left(1 - \frac{1}{\text{OPT}}\right)^t < \frac{1}{n}$$

Taking logarithms:

$$t \ln\left(1 - \frac{1}{\text{OPT}}\right) < -\ln n$$

Since $\ln(1 - \frac{1}{\text{OPT}}) < 0$, dividing reverses the inequality:

$$t > \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)}$$

Let $S$ denote the size of the dominating set produced by the greedy algorithm. Then $S$ is the smallest integer $t$ satisfying the above inequality. Thus:

$$S = \left\lceil \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} \right\rceil \leq \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} + 1$$

Now using the inequality

$$-\ln(1 - x) \geq x$$

with $x = \frac{1}{\mathrm{OPT}}$, we obtain

$$-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right) \geq \frac{1}{\mathrm{OPT}}$$

Taking reciprocals reverses the inequality:

$$\frac{1}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} \leq \mathrm{OPT}$$

$$\implies S \leq \ln n \cdot \frac{1}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} + 1$$

$$\implies S \leq \mathrm{OPT} \ln n + 1$$

Hence the greedy algorithm produces a dominating set of size at most

$$\boxed{\mathrm{OPT} \ln n + 1}$$

## 9.2 Independent Set

**The Problem** We are given a set of exams and a set of students. Each student is enrolled in a subset of the exams. We wish to schedule as many exams as possible in the first time slot[1], subject to the constraint that no student should have two exams scheduled at the same time[2].

The goal is to find a largest possible subset of exams that can be scheduled simultaneously without any conflict.

### Reduction

We model the problem using a graph $G = (V, E)$ where:

- each vertex represents an exam,
- an edge exists between two vertices if there is at least one student enrolled in both corresponding exams.

In this formulation, a feasible schedule corresponds to a set of vertices with no edges between them. Such a set is called an *independent set*. Thus, the problem reduces to finding a maximum independent set in a graph.

### 9.2.1 Random Permutation

$V$ is the set of all vertices. Maintain a set - the independent set we are building ($S$). Pick a random vertex from $V$ – delete it from $V$ and if none of its neighbours are in $S$, put it in $S$. Keep doing this until $V$ is empty. This guarantees forming a valid independent set.

Note that the order in which we pick vertices is a uniformly random permutation of the vertices.

[1] this is different from the problem of the overall scheduling problem which is equivalent to finding the Chromatic Number, i.e. the largest number of colors needed to colour a graph which is equivalent to the minimum number of slots required to conduct the whole exam schedule. This is a simpler problem where we only care about the first time slot.

[2] Or you could just give them a Time Turner and hope for the best :)

### Probability a Vertex is Selected

Fix a vertex $v$. Consider the set consisting of $v$ and its neighbours $N(v)$. There are $d(v) + 1$ vertices in total.

Since all relative orderings are equally likely, the probability that $v$ is the picked earliest among these vertices is:

$$\Pr[v \in S] = \frac{1}{d(v) + 1}$$

### Expected Size of the Independent Set

Let $I_v$ be the indicator random variable for the event $v \in S$. Then:

$$|S| = \sum_{v \in V} I_v$$

Taking expectation and using linearity:

$$\mathbb{E}[|S|] = \sum_{v \in V} \mathbb{E}[I_v] = \sum_{v \in V} \frac{1}{d(v) + 1}$$

Therefore, there exists an independent set of size at least:

$$\sum_{v \in V} \frac{1}{d(v) + 1}$$

Denoting the maximum independent set size by $\alpha(G)$, we have:

$$\alpha(G) \geq \sum_{v \in V} \frac{1}{d(v) + 1}$$

Using the Jensen's[3] inequality on 1/(x+1), we have:

$$\sum_{v \in V} \frac{1}{d(v) + 1} \geq \frac{n}{d + 1}$$

where $d$ is the average degree of the graph. Hence:

$$\alpha(G) \geq \frac{n}{d + 1}$$

## 9.2.2 Local Decision

We now describe an alternative randomized construction that produces a large independent set via local random choices.

Each vertex independently chooses a random value:

$$X_v = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

We construct a set $S$ by including a vertex $v$ if:

- $X_v = 1$, and
- for every neighbour $u \in N(v)$, $X_u = 0$.

[3] Refer Appendix B.2

### Probability of Inclusion

Fix a vertex $v$. The probability that $v$ is included in $S$ is:

$$\Pr[v \in S] = p(1-p)^{d(v)}$$

Therefore:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}$$

### Guarantee on Expected Size

We analyze the randomized construction where each vertex is selected independently with probability $p$, and a vertex $v$ is included in the independent set $S$ if it is selected and none of its neighbors are selected. Then:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}.$$

Using convexity of $f(x) = (1-p)^x$ and Jensen's inequality, we obtain:

$$\sum_{v \in V} (1-p)^{d(v)} \geq n(1-p)^d,$$

where $d = \frac{1}{n}\sum_{v \in V} d(v)$ is the average degree. Hence:

$$\mathbb{E}[|S|] \geq np(1-p)^d.$$

We now optimize the function:

$$f(p) = p(1-p)^d.$$

Differentiating:

$$f'(p) = (1-p)^d - pd(1-p)^{d-1} = (1-p)^{d-1}\big((1-p) - pd\big).$$

Setting $f'(p) = 0$, we obtain:

$$1 - p - pd = 0 \quad \Rightarrow \quad p = \frac{1}{d+1}.$$

Substituting this optimal value gives:

$$\mathbb{E}[|S|] \geq n \cdot \frac{1}{d+1}\left(1 - \frac{1}{d+1}\right)^d = \frac{n}{d+1}\left(\frac{d}{d+1}\right)^d.$$

Using the standard bound $\left(1 - \frac{1}{d+1}\right)^d \geq e^{-d/(d+1)}$, we obtain:

$$\mathbb{E}[|S|] \geq \frac{n}{d+1} \cdot e^{-d/(d+1)}. \geq \frac{n}{e(d+1)}.$$

This is a slightly weaker bound than the previous method, but it has the advantage of being local and easily parallelizable, as each vertex makes its decision independently based on local information.

Recall from the chapter on data structures that in Skip Lists we let each element locally decide how many levels to appear in. The reason for that was updation was quick since we did not have to change everybody's decision just because of the addition of a single element.

A similar reasoning can be applied here to justify the utility of a local decision algorithm.

## 9.3 Bipartite Subgraph Revisited

Recall the problem of finding a bipartition of the vertices that maximizes the number of bichromatic edges. We showed that a random coloring achieves at least $m/2$ bichromatic edges in expectation, and with some probability. Now we refine our approach by showing that we can always find a coloring with at least $m/2$ bichromatic edges via a self-correction procedure.

### 9.3.1 Self-Correction

Start with a random coloring of the vertices. If it has at least $m/2$ bichromatic edges, we are done.

Otherwise, let $X$ be the number of bichromatic edges. We have $X < m/2$, so there are more monochromatic edges than bichromatic edges. Call a vertex $v$ *bad* if it has more monochromatic incident edges than bichromatic incident edges. Otherwise it is *good*.

**Claim:** If the current cut has fewer than $m/2$ crossing edges, then a bad vertex must exist.

**Proof idea:** Each monochromatic edge contributes equally to two endpoints. So total number of monochromatic edges is half the sum of monochromatic degrees of vertices. If globally there are more monochromatic edges than crossing ones, by pigeonhole principle[4], some vertex must be incident to more monochromatic than bichromatic edges.

Now consider flipping the color of a bad vertex $v$. Let $d_h(v)$ be the number of monochromatic (same-color) edges incident to $v$ and $d_c(v)$ the number of bichromatic edges. Since $d_h(v) > d_c(v)$, flipping $v$ increases $X$ by

$$\Delta X = d_h(v) - d_c(v) > 0.$$

Thus, each flip strictly increases the number of bichromatic edges. This means that the number of steps is upper bounded by $m/2$.

**Pseudocode:**

```
initialize random coloring of vertices
compute X

while exists bad vertex v:
    flip color of v
    update X locally
```

**Complexity analysis:** Each flip increases $X$ by at least 1, and $X \leq m$, so there are at most $m$ flips.

If we maintain for each vertex the counts of monochromatic and bichromatic incident edges, each flip can be updated in $\Theta(\deg(v))$. To find a bad vertex we need atmost $\Theta(n)$ time. Over all flips, total time is atmost

$$O\left(\sum_{v \text{ flipped}} (\deg(v) + n)\right) = \Theta(mn + m).$$

Hence the full procedure runs in atmost [5]$\Theta(mn + m)$ after initialization.

[4] You may also prove this by contradiction – if all vertices are bad, then the total number of monochromatic edges is more than the total number of bichromatic edges by summing the inequality, contradicting the assumption.

[5] this can be improved to $\Theta(m \log n)$ by using priority queues with lazy updates – its a fun competitive programming exercise, I encourage you to try it!

# Appendices

# Appendix A

# Probability and Expectation

## A.1 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (not necessarily independent),

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

## A.2 Tail Sum Formula

For a non-negative integer-valued random variable $X$,

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} \Pr(X \geq k) = \sum_{k=0}^{\infty} \Pr(X > k)$$

For a non-negative continuous random variable $X$,

$$\mathbb{E}[X] = \int_0^{\infty} \Pr(X \geq t)\, dt$$

## A.3 Union Bound

For any events $A_1, A_2, \ldots, A_n$,

$$\Pr\left(\bigcup_{i=1}^{n} A_i\right) \leq \sum_{i=1}^{n} \Pr(A_i)$$

This is often used to upper bound the probability that at least one "bad event" occurs.

## A.4 Independence of Expectation

If $X$ and $Y$ are independent random variables, then

$$\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$$

## A.5 Variance

For a random variable $X$, define the variance of $X$ as

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

For any two random variables $X$ and $Y$,

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y) \quad (\text{where } \mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y])$$

If $X$ and $Y$ are independent, then $\mathrm{Cov}(X,Y) = 0$, so

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$$

More generally, for any collection $\{X_1, \ldots, X_n\}$,

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2 \sum_{1 \le i < j \le n} \mathrm{Cov}(X_i, X_j)$$

## A.6 Indicator Variables

For an event $A$, define the indicator random variable

$$I_A = \begin{cases} 1 & \text{if } A \text{ occurs,} \\ 0 & \text{otherwise.} \end{cases}$$

$$\implies \mathbb{E}[I_A] = \Pr(A)$$

## A.7 Law of Total Expectation

Let $X$ and $Y$ be random variables. Then

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$$

provided the expectations exist. Intuitively, the theorem states that we may first compute the expected value of $X$ under every possible circumstance described by $Y$, and then average those conditional expectations according to the probability of each $Y$.

For a series of random variables $X_0, X_1, \ldots,$

$$\mathbb{E}[X_{i+1}] = \mathbb{E}[\mathbb{E}[X_{i+1} \mid X_i]]$$

## A.8 Wald's Equation

### Definitions

Let $X_1, X_2, \ldots, X_N$ be i.i.d. random variables with finite expected value $\mu$.

$$\mathbb{E}[X_1] = \mathbb{E}[X_2] = \cdots = \mathbb{E}[X_N] = \mu$$

Now let the number of $X$'s in the series (which is of course, $N$) also be a random variable with expected value $n$.

### Valid Stopping Time

For $N$ to be a valid stopping time for a series of random variables $X_1, X_2, \ldots$ the decision of whether to stop at $i$ (implying $N = i$) must only depend on the information gathered until $X_i$. This is fairly intuitive since the decision to stop can not be based of off future information.

For example, a rule like "$N = i$ if $X_i < \mu$" is valid but the rule "$N = i$ if $X_{i+1} > \mu$" is not.

### The Equation

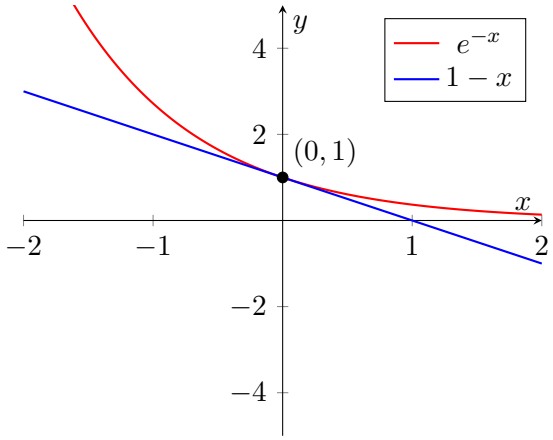
Given that $N$ is a valid stopping time for the series of random variables $X_1, X_2, \ldots$ the Wald's Equation states that the expected sum equals the expected number of terms multiplied by the expected value of each term.

$$\mathbb{E}\left[\sum_{i=1}^{N} X_i\right] = \mathbb{E}[N] \cdot \mathbb{E}[X] = n \cdot \mu$$

# Appendix B

# Inequalities

## B.1 Exponential Inequality




$$1 - x \leq e^{-x} \ \forall \ x \in \mathbb{R} \implies (1 - x)^k \leq e^{-kx} \ \forall \ k \in \mathbb{N}, x \in \mathbb{R}$$

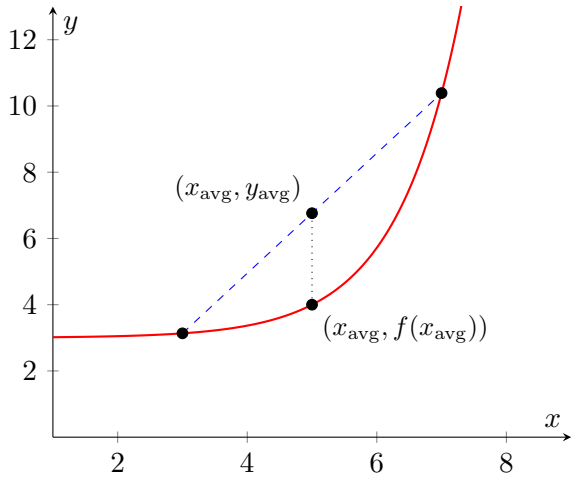
## B.2 Jensen's Inequality

For a concave-up function $f$ and random variable $X$,

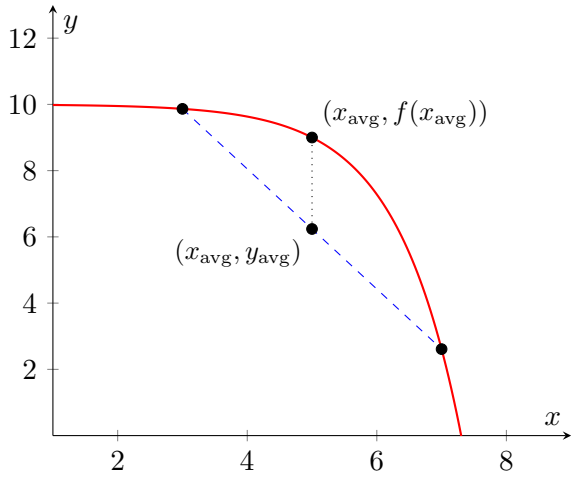
$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$

Another form of Jensen's inequality is for a convex function $f$ and real numbers $x_1, x_2, \ldots, x_n$,

$$f(\frac{1}{n}\sum_{i=1}^{n} x_i) \leq \frac{1}{n}\sum_{i=1}^{n} f(x_i) \iff f(x_{avg}) \leq y_{avg}$$




For concave-down functions, the inequality is reversed, i.e. $f(x_{avg}) \geq y_{avg}$.




## B.3 Markov's Inequality

Let $X$ be a non-negative random variable and let $a > 0$. Then

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

### Proof

Define the indicator variable $I$ as

$$I = \begin{cases} 1 & \text{if } X \geq a, \\ 0 & \text{otherwise.} \end{cases}$$

Then $X \geq aI$ follows by definition! Taking expectations,

$$\mathbb{E}[X] \geq a\,\mathbb{E}[I] = a\,\Pr(X \geq a) \implies \Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

An equivalent and useful form is $Pr(X \geq c\,\mathbb{E}[X]) \leq \frac{1}{c}$

# Appendix C

# Convergence and Limit Theorems

It is quite difficult to properly grasp what these theorems say if you do not know the precise definitions of the terms used to state them. That is why I decided to be more elaborate in this appendix because there a lot of common misconceptions which I struggled to get clarity on until I took a rigorous and exhaustive approach.

## C.1 Identical and Independently Distributed Random Variables

This is also denoted by i.i.d. for short. If it is stated that $X_1, X_2, \ldots$ are i.i.d, then they are independent and have the same probability distribution. They are essentially copies of each other, but the value of one does not affect the others.

## C.2 Almost Surely

When we say an event occurs "almost surely" we mean that it is true with probability one. The phrase "almost surely" can be used within a sentence like "X is almost surely equal to Y" which means "X is equal to Y" almost surely. We can also say that an event being almost sure is equivalent to the event having "almost every" outcome in its set.

This does not necessarily mean that every outcome is part of the event, it just means the "measure" of the set of outcomes in the event is equal to the total measure of the sample space. We will not be rigorously defining measure here but let me demonstrate via an example.

A random real number chosen from $[0, 1]$ is almost surely not $\frac{1}{2}$. This is because the measure of the set $[0, 1] - \{\frac{1}{2}\}$ is equal to the measure of the sample space.

## C.3 Almost Never

When we say an event occurs "almost never" we mean that it is true with probability zero. Again, this does not mean that the event has no outcomes in its set, but rather that the measure of that set is zero as compared to the measure of the sample space. The complementary event of an almost never event is an almost sure event.

The example we used earlier can be restated as "a random real number chosen from $[0, 1]$ is almost never $\frac{1}{2}$".

## C.4 $\lim_{n\to\infty} X_n = Y$

The event $\lim_{n\to\infty} X_n = Y$ is defined (where all $X_n$ and $Y$ are random variables) as the set of all outcomes $\omega$ such that $\lim_{n\to\infty} X_n(\omega) = Y(\omega)$. Since the sequence $X_n(\omega)$ is a real number sequence, the usual definition of limit applies.

In case $Y$ is a constant $a$ then it is the set of all outcomes $\omega$ such that $\lim_{n\to\infty} X_n(\omega) = a$. I personally do not like this notation since we are trying to define the notion of convergence and hence limits for random variables and in the definition we are using the notation $\lim_{n\to\infty} X_n$ which makes it seem circular when it really is not. The notation means to talk about the limit of sequences of real numbers and not a sequence of random variables.

## C.5 Convergence in Probability

A sequence of random variables $X_n$ "converges in probability" to a constant $a$ if for every $\varepsilon > 0$, the probability of $X_n$ deviating from $a$ by more than $\varepsilon$ tends to zero.

$$X_n \xrightarrow{\text{P}} a \iff \lim_{n\to\infty} \Pr(|X_n - a| > \varepsilon) = 0 \ \ \forall\, \varepsilon > 0$$

Note that we can **not** in general, swap the limit and the probability.

## C.6 The Weak Law of Large Numbers

Let $X_1, X_2, \ldots, X_n$ be i.i.d random variables with $\mathbb{E}[X_i] = \mu$ and finite variance $\sigma^2$. Define the random variable $Z_n$ to be average of the random variables $X_1, X_2, \ldots, X_n$.

$$Z_n = \frac{X_1 + X_2 + \cdots + X_n}{n}$$

The Weak Law of Large Numbers states that $Z_n$ converges in probability to $\mu$.

$$n \to \infty \implies Z_n \xrightarrow{P} \mu$$

### Proof

For independent random variables, the variance of their sum is the sum of the variances.

$$\text{Var}(X_1 + \cdots + X_n) = n\sigma^2$$

Dividing by $n^2$ yields the variance of the average –

$$\text{Var}(Z_n) = \frac{\sigma^2}{n}$$

Applying Chebyshev's inequality (only valid if the variance is finite) gives us the probability of a deviation of the average from its expected value –

$$\Pr(|\overline{Z}_n - \mu| > \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \implies \lim_{n\to\infty} \Pr(|Z_n - \mu| > \varepsilon) \leq \lim_{n\to\infty} \frac{\sigma^2}{n\varepsilon^2} = 0$$

So the probability that $Z_n$ deviates from $\mu$ by more than $\varepsilon$ tends to zero for all $\varepsilon > 0$

$$\lim_{n\to\infty} \Pr(|Z_n - \mu| > \varepsilon) = 0$$

This is equivalent to $Z_n$ converging in probability to $\mu$, hence the Weak Law of Large Numbers is proved.

## C.7 Almost Sure Convergence

A sequence of random variables $X_n$ "almost surely converges" to a constant $a$ if $\lim_{n\to\infty} X_n = a$ almost surely. This means for almost every outcome $\omega$, $\lim_{n\to\infty} X_n(\omega) = a$.

$$X_n \xrightarrow{\text{a.s.}} a \iff \Pr\Big(\lim_{n\to\infty} X_n = a\Big) = 1$$

## C.8 $A_n$ infinitely often

Let $A_1, A_2, A_3, \ldots$ be a sequence of events. Then the event "$A_n$ infinitely often" or "$A_n$ i.o." is defined as there are infinite $n \in 1, 2, 3, \ldots$ such that $A_n$ occurs. Thus the event "$A_n$ i.o." occurs iff for all N, at least one of the events $A_N, A_{N+1}, A_{N+2}, \ldots$ occur.

$$A_n \text{ i.o. } = \bigcap_{N=1}^{\infty} \bigcup_{n=N}^{\infty} A_n$$

## C.9 The Borel–Cantelli Lemmas

The **first** Borel–Cantelli lemma states that if the sum of $Pr(A_n)$ is finite, then $A_n$ i.o. almost never.

$$\sum_{n=1}^{\infty} \Pr(A_n) < \infty \implies \Pr(A_n \text{ i.o.}) = 0$$

The **second** Borel–Cantelli lemma states that if all $A_n$ are independent and the sum of $Pr(A_n)$ diverges, then $A_n$ i.o. almost surely.

$$\text{All } A_n \text{ independent and } \sum_{n=1}^{\infty} \Pr(A_n) = \infty \implies \Pr(A_n \text{ i.o.}) = 1$$

## Example

It is not yet intuitively obvious what is the difference between almost sure convergence and convergence in probability. Consider the sequence of independent random variables $X_1, X_2, \ldots$ such that

$$X_n = \begin{cases} 1 & \text{with probability } \frac{1}{n} \\ 0 & \text{otherwise} \end{cases}$$

Let us consider the convergence in probability of $X_n$ to 0. For $\varepsilon \geq 1$, $Pr(|X_n - 0| > \varepsilon) = 0$ since $X_n$ is at most 1 thus $|X_n - 0| \leq 1$. For $\varepsilon < 1$, $Pr(|X_n - 0| > \varepsilon) = \frac{1}{n}$. In both cases, as $n \to \infty$, $Pr(|X_n - 0| > \varepsilon) \to 0$. Hence $X_n$ converges in probability to 0.

Now consider the almost sure convergence to 0 – the probability that the event $(X_n \neq 0)$ occurs is $\frac{1}{n}$ for all $n$. Applying the second Borel–Cantelli lemma on the event $(X_n \neq 0)$ we get $Pr(X_n \neq 0 \text{ i.o.}) = 1$.

$$\sum_{n=1}^{\infty} \frac{1}{n} = \infty \implies Pr(X_n \neq 0 \text{ i.o.}) = 1$$

Thus the sequence $X_n(\omega)$ almost surely has infinitely many $n$ where $X_n(\omega) \neq 0$. From the concept of limits of real number sequences, a sequence of 0's and 1's has infinitely many 1's iff it can not converge to 0. Since it almost surely has infinitely many 1's, it almost never converges

to 0.

So this properly disproves the almost sure convergence of the sequence $X_n$ to 0. In fact, it is almost never convergent to 0, yet it is convergent in probability to 0. Hence the two notions are not the same.

With some thought, it can be shown that almost sure convergence implies convergence in probability, but as we saw, not necessarily the other way around.

## C.10 The Strong Law of Large Numbers

Let $X_1, X_2, \ldots, X_n$ be i.i.d random variables with $\mathbb{E}[X_i] = \mu$ and finite variance $\sigma^2$. Define the random variable $Z_n$ to be average of the random variables $X_1, X_2, \ldots, X_n$.

$$Z_n = \frac{X_1 + X_2 + \cdots + X_n}{n}$$

The Strong Law of Large Numbers states that $Z_n$ converges almost surely to $\mu$.

$$X_n \xrightarrow{\text{a.s.}} \mu$$

I feel there is not much utility at this point in stating the proof. We should focus on understanding the result with an application!

## Application

If I want to find the expected value of a random variable $X$ and the only thing I can do is sample the variable $X$ ... then I can approximate the expected value really well by averaging over all the samples. This is because sampling a random variable $X$ is equivalent to creating a random variable $X_1$ and then evaluating it at a random outcome $\omega$. Sampling it $n$ times is equivalent to creating $n$ i.i.d. random variables $X_1, X_2, \ldots, X_n$ and then evaluating them at a random outcome $\omega$.

Taking the average on the n samples would mean evaluating the average random variable $Z_n$ at $\omega$ –

$$Z_n(\omega) = \frac{X_1(\omega) + X_2(\omega) + \cdots + X_n(\omega)}{n}$$

Assuming n really is very large, we can use the Strong Law of Large Numbers to state that $Z_n$ converges almost surely to the expected value of $X_1$. This is equivalent to saying for almost every outcome $\omega$, $Z_n(\omega)$ converges to $\mathbb{E}[X_1]$. In practice for well-behaved random variables this does work quite well if the variance is not too large. It is usually used in physics for experimental data.

If we want more guarantees about the distribution of the average $Z_n$, we can use the Central Limit Theorem.

## C.11 Convergence in Distribution

Let $X_n$ be a sequence of random variables with distribution functions $F_n(x) = \Pr(X_n \leq x)$. We say that $X_n$ converges in distribution to a random variable $X$, written as

$$X_n \xrightarrow{d} X,$$

if the distribution functions satisfy

$$F_n(x) \to F(x)$$

This is the weakest form of convergence. Convergence in probability implies convergence in distribution.

## C.12 The Central Limit Theorem

If $X_1, X_2, \ldots$ are i.i.d. with expected value $\mu$ and finite variance $\sigma^2$ and $Z_n$ be their average, then

$$\frac{\sqrt{n}}{\sigma}(Z_n - \mu) \xrightarrow{d} \mathcal{N}(0,1)$$

This means that the distribution of the "normalized" average approaches the standard normal distribution as $n \to \infty$, even though the average itself approaches $\mu$. Here the normalisation factor is $\frac{\sqrt{n}}{\sigma}$. Interestingly, the Central Limit Theorem is enough to prove the Weak Law of Large Numbers.

What can we conclude from the Central Limit Theorem? We can say that the average of a large number of samples have fluctuations from the mean of order $\frac{\sigma}{\sqrt{n}}$. In other words the error in the average is $\Theta(\frac{1}{\sqrt{n}})$. Since we know the approximate distribution of the average, we can find things like the value of the probability of it deviating from the expected value by some $\varepsilon$ instead of using weak general upper bounds like Chebyshev.

# Appendix D

# Discrete Mathematics

## D.1 Pigeonhole Principle

Let $a_1, a_2, \ldots, a_n$ be non-negative real numbers and let

$$S = \sum_{i=1}^{n} a_i, \text{ then there } \textit{exists} \text{ an } i \text{ such that } a_i \geq \frac{S}{n}$$

This simply states that at least one value is at least the average value.

### Proof by Contradiction

If every $a_i < \frac{S}{n}$, then summing over all $i$ gives

$$\sum_{i=1}^{n} a_i < S$$

which is a contradiction.

## D.2 Graph Theory

A graph $G$ is an ordered pair $(V, E)$ where $V$ is a set of vertices and $E \subseteq \{\{u, v\} \mid u, v \in V,\ u \neq v\}$ is a set of edges. $|E|$ is usually denoted by $m$ and $|V|$ is usually denoted by $n$.

### Degree

For a vertex $v \in V$, the degree $d(v)$ is the number of edges which have $v$ as an endpoint. Equivalently, $d(v)$ is the number of neighbours of $v$.

Given n is the number of vertices, the average degree of a graph is

$$d = \frac{1}{n}\left(\sum_{v \in V} d(v)\right) = \frac{2m}{n}$$

### Bipartite Graph

A graph $G = (V, E)$ is bipartite if the vertex set $V$ can be partitioned into two disjoint sets $L$ and $R$ such that

$$V = L \cup R, \quad L \cap R = \emptyset$$

and every edge connects a vertex in $L$ to a vertex in $R$. That is, there are no edges with both endpoints in the same set.

### Subgraph

A subgraph of a graph $(V, E)$ is a pair of sets $(V', E')$ where $V' \subseteq V$ and $E' \subseteq E$ with the additional condition that the subgraph must be a valid graph!

# Appendix E

# Linear Algebra

## E.1 Vector Spaces

Let $\mathbb{F}$ be a field. A vector space over $\mathbb{F}$ is a set $V$ equipped with vector addition and scalar multiplication satisfying the usual axioms:

- closure under addition and scalar multiplication,
- associativity and commutativity of addition,
- existence of a zero vector,
- existence of additive inverses,
- distributive properties,
- compatibility of scalar multiplication.

An example of a common vector space is the set of $n$-tuples of elements from $\mathbb{F}$,

$$\mathbb{F}^n = \{(x_1, x_2, \ldots, x_n) : x_i \in \mathbb{F}\}$$

Common choices for the field $\mathbb{F}$ include the real numbers $\mathbb{R}$ or the integers $\mathbb{Z}$ with the usual addition and multiplication, and the finite field $\mathbb{F}_p$ where $p$ is a prime with addition and multiplication modulo $p$.

## E.2 Subspaces

A subset $W \subseteq V$ is called a subspace of the vector space $V$ if

1. $0 \in W$
2. for all $u, v \in W$, we have $u + v \in W$
3. for all $u \in W$ and $\alpha \in \mathbb{F}$, we have $\alpha u \in W$

## E.3 Dimension of a Subspace

A collection of vectors $v_1, v_2, \ldots, v_k$ is called a basis of a subspace $W$ if

- the vectors are linearly independent
- any vector in W is a linear combination of the vectors

The number of vectors in any basis of $W$ is called the dimension of $W$ and is denoted by $\dim(W)$.

## E.4 The Nullspace

Let $M \in \mathbb{F}^{m\times n}$. The nullspace of $M$ is defined by

$$\mathcal{N}(M) = \{x \in \mathbb{F}^n : Mx = 0\}.$$

The nullspace is also called the *kernel* of $M$, and we write

$$\ker(M) = \mathcal{N}(M).$$

The nullspace of a matrix is a subspace of $\mathbb{F}^n$.

## E.5 Nullity and Rank

Since the nullspace is a subspace, it has a well-defined dimension.

Define $\text{nullity}(M) = \dim(\ker(M))$.

The rank of $M$, denoted $\text{rank}(M)$ is the dimension of the column space or row space of $M$, equivalently the maximum number of linearly independent columns or rows of $M$.

## E.6 The Rank–Nullity Theorem

One of the fundamental results of linear algebra is the rank–nullity theorem.

**Theorem (Rank–Nullity).** For every matrix $M \in \mathbb{F}^{m\times n}$, we have

$$\text{rank}(M) + \text{nullity}(M) = n.$$

An immediate consequence is that if $M \neq 0$, then

$$\text{rank}(M) \geq 1, \implies \text{nullity}(M) \leq n - 1.$$